Base de Dados

Grupo P1 & 10

-66:14 24 708

Nova possi: matos2069

Descrição lógica dos dados na BD chama-se de esquema (schema)

1º Geração de bases de dados

Modelo Hierárquico - Os dados são orate nodos em árvore, um registo pai é associado a N registos

1:N, a operação de procura contrária faz com que se tenha de percorrer a árvore toda, também leva a replicação de inf. em casos de o mesmo nod estar associado a dif. ramos.

Este modelo é útil em sistemas de acesso sequencial.

~~No entanto não~~

Mas para realizar pesquisa inversa trás grandes desvantagens pois teria de haver uma árvore inversa, daí replicação de dados.

Modelo de Rede

O mesmo registo está associado a vários nós e a partir de um é possível alcançar outros, não é necessário percorrer toda a base de dados para a encontrar.

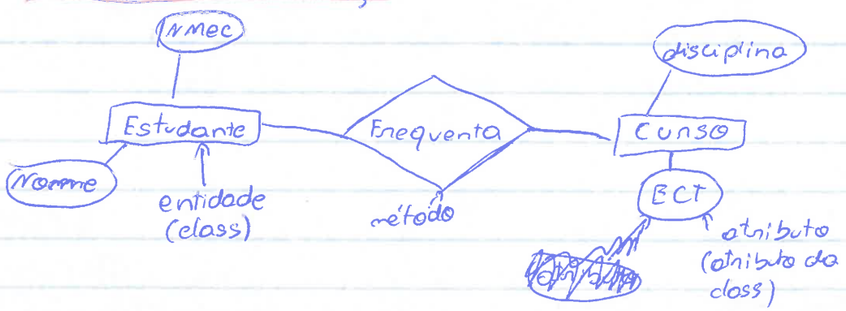
Desenhar Base de Dados

- Análise de Requisito
- Mode E/R -> diagrama elustrativo das funções
- Esquema lógico

⋮

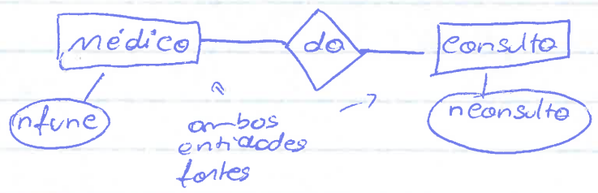
Análise de Requisitos — Levantamento detalhado de toda a inf. associado ao problema.
- Observação
- Compreender o problema
- Filtrar os dados
- Discussão de aspetos dúbios, falhas

Modelo Entidade/Relação



- Entidades Fortes → têm um atributo que os identifica

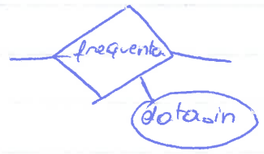
- " Fraca → Para identificá-la é necessário ~~se~~ associá-la a outra classe



Num cenário em que o nº de consulta é feito por médico, por exemplo: cada médico tem a consulta 1, 2, 3, ...

assim há redundância que inf. e a único maneira de identificar uma consulta, é associá-la a ~~um~~ médico, o médico é uma entidade forte e as consultas são entidades fracas

Relações



classificação de relações
- Grau - nº de entidades
- Obrigatoriedade - uma entidade pode ou não ser obrigado a participar numa relação

~~Nota~~

~~relações recursivas:~~
- ~~assimétrica não interessa~~ Cardinalidade

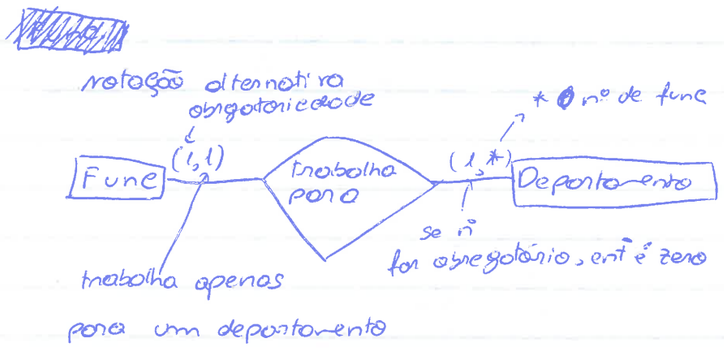


- Um médico tem N consultas mas as consultas apenas têm um médico

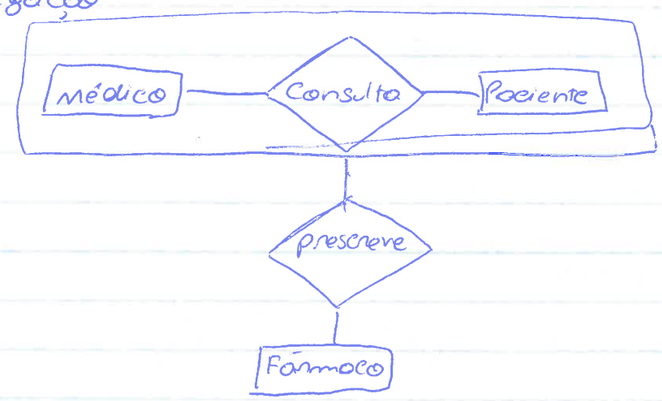
Notação de chen



Um funcionário gere um departamento



Agregação



1º conjunto de slides

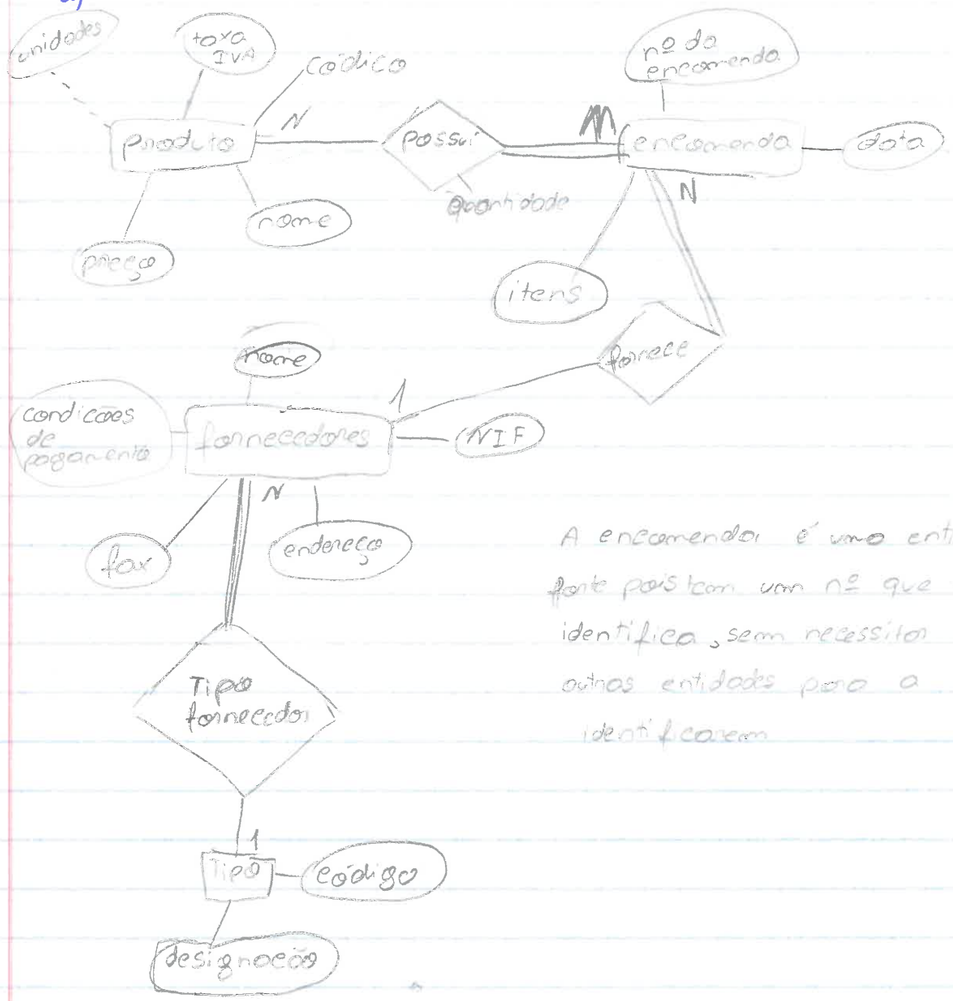
Slide 28

A opção A é mais flexível, um empréstimo pode estar associado a M clientes, enquanto que a B apenas permite a associação a um cliente

1  fazer a modelação da empresa

Problema 2.1

a)



A encomenda é uma entidade forte pois tem um nº que a identifica, sem necessitar de outras entidades para a identificarem

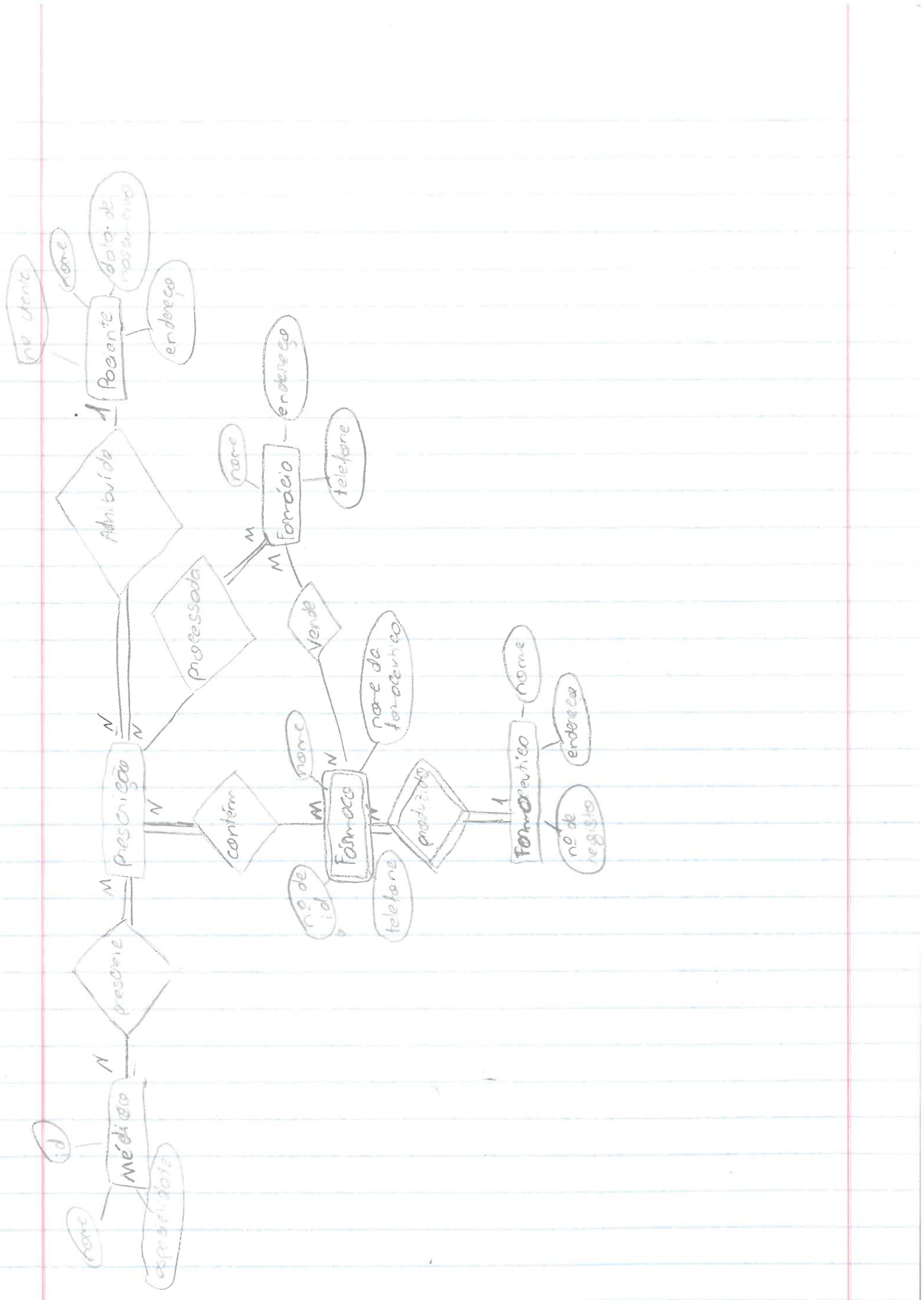
Problema 2.2

Entidades:

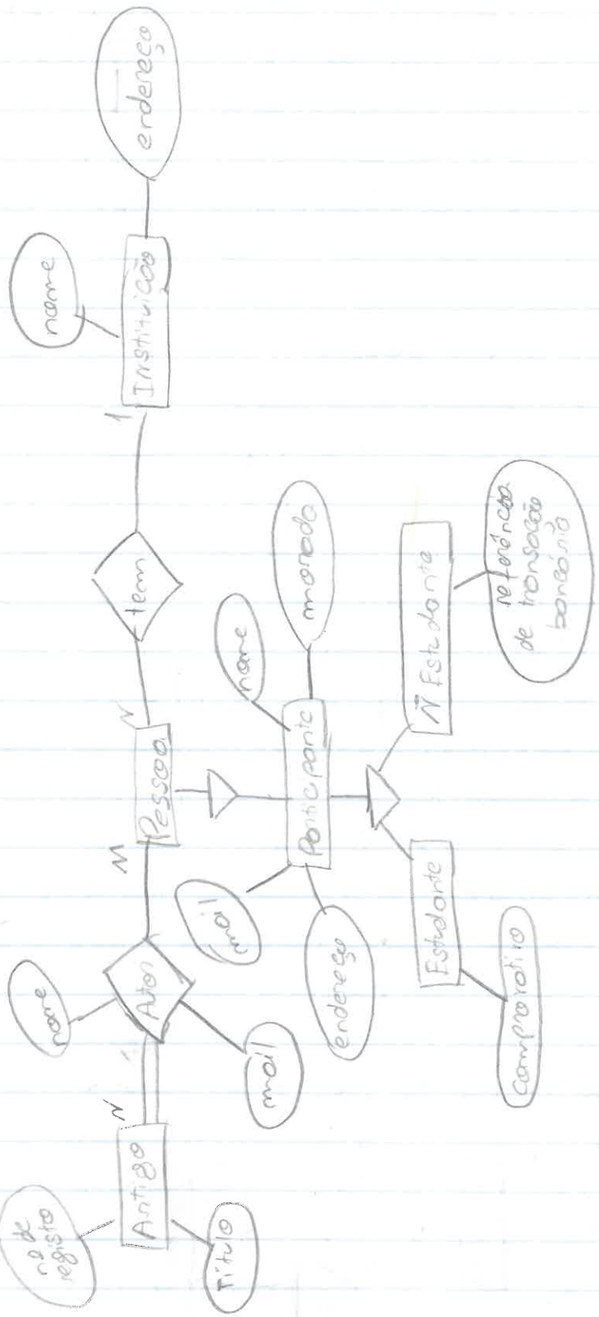
- Prescrição
- Fármaco
- Companhia
- Médico
- Farmácia
- Paciente

Atributos:

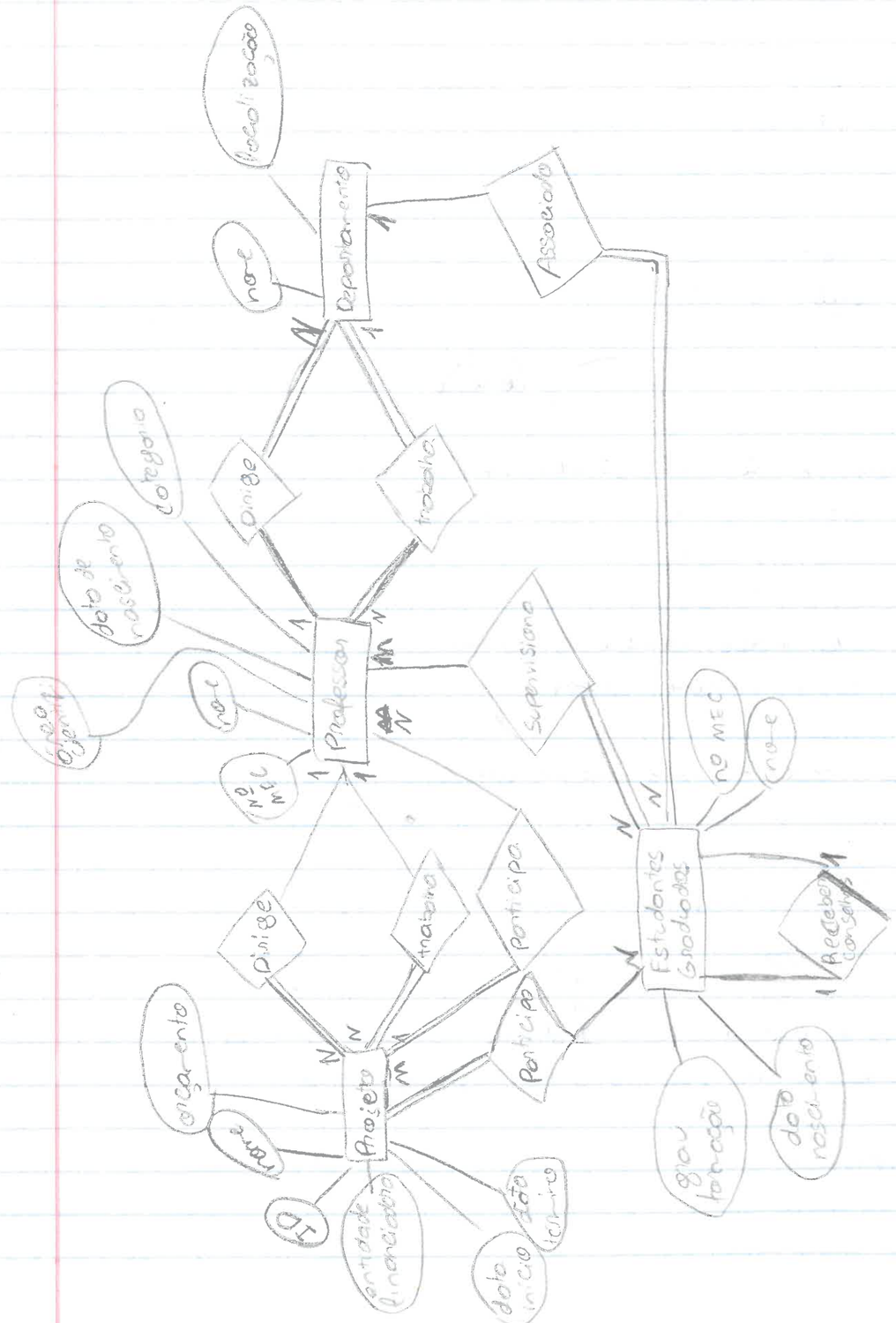
- Prescrição
  - nº da prescrição
- Fármaco
  - nome comercial
  - companhia
  - nome único para essa empresa
- Companhia
  - registo nacional
  - nome
  - endereço
- Médico
  - nº de id
  - nome
  - especialidade
- Farmácia
  - nome
  - endereço
  - telefone
- Paciente
  - nº de utente
  - nome
  - data de nascimento



Problema 2.3



Problema 2.4



Modelo Relacional

Todos os bases de dados usam um modelo relacional, este tendência tornou-se popular nos 90

Recomenda-se primeiro aprender Modelo relacional e de seguida Information Retreard.

O modelo relacional baseia-se na teoria dos conjunto usa a noção "relações"

estrutura de dados onde ñ há repetição de dados

Criamos uma Matriz (Tabelas) em que cada instância é um registo

Esquema da relação → Nome do esquema com a lista de Atributos
Ex: pessoa(nome, bi, idade)

Relação chaves → uma ~~relo~~ maneira de identificar ou relacionar elementos

Chaves → elementos que identifique de for única uma instância da tabela

Sabemos que o conjuntos dos atributos identificam de forma única um elemento pois sabemos que ñ há repetições

Há sempre uma superchave, que é o conjuntos do atributos.

Exemplo:

Estudante(Nome, mail, NMEC, Curso)

Superchaves:

Nome, Email, NMEC
Nome, NMEC
⋮
NMEC
Email

NMEC → subconjuntos
Email → que ñ perdem as características de superchave

⇓

Chave Candidata a primária

Devemos ter ~~em~~ atenção a alguns fatores

A escolha da chave primária depende de fatores como o facto de futuramente ~~a~~ mudar, o email pode mudar, uma pessoa pode mudar o email.

Assim o NMEC torna-se uma chave primária e o email, uma chave única

Chaves Estrangeiras - ~~S~~ chaves primárias de tabelas que ~~S~~ usadas noutras tabelas, que ñ a tabela em que é primária, para identificar algum ele.

Restrição de Integridade → Tema muito importante

- Domínio
- Entidade

Umas das restrições no domínio que se pode efectuar é obrigar uma relação

Ex um Trabalhador tem que trabalhar no departamento, então obriga o atributo Nº Tabelo de Departo. em que trabalho ñ pode estar null

Regras Codd

Verificam se o sedimente é possível chamar de relacional uma base dados.

12 ✓

4 - Os metadados (informação de como é acedidos e armazenados os dados) devem ser armazenados da mesma que outros dados são.

Passos do modelo Entidade para mod. relacional

usar as entidades fortes para criar tabelas

Para as entidades fracas usa-se a chave primárias das entidades dominantes como chave estrangeira

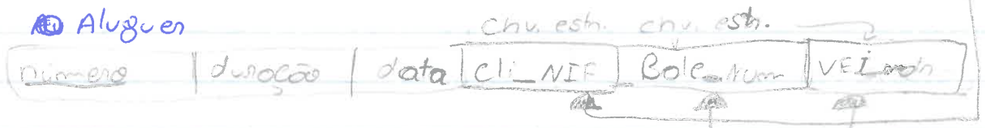
Nos caso 1:1 sem nenhuma participação obrigatória, também se cria uma tabela para o relacionamento.

P3.1

Cliente



Aluguer



Balcão



Veículo



Tipo_veículo



Ligeiro



Pesados

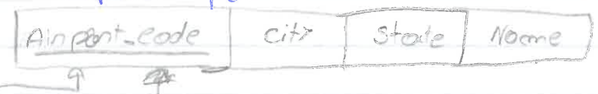


Similaridade



P3.2

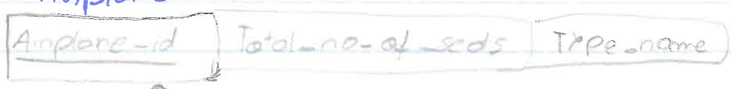
~~Airport~~ Airport



Airplane_Type



Airplane



Flight



Flight_LEG



FARE



Leg_Instance



~~Seats~~ Seats

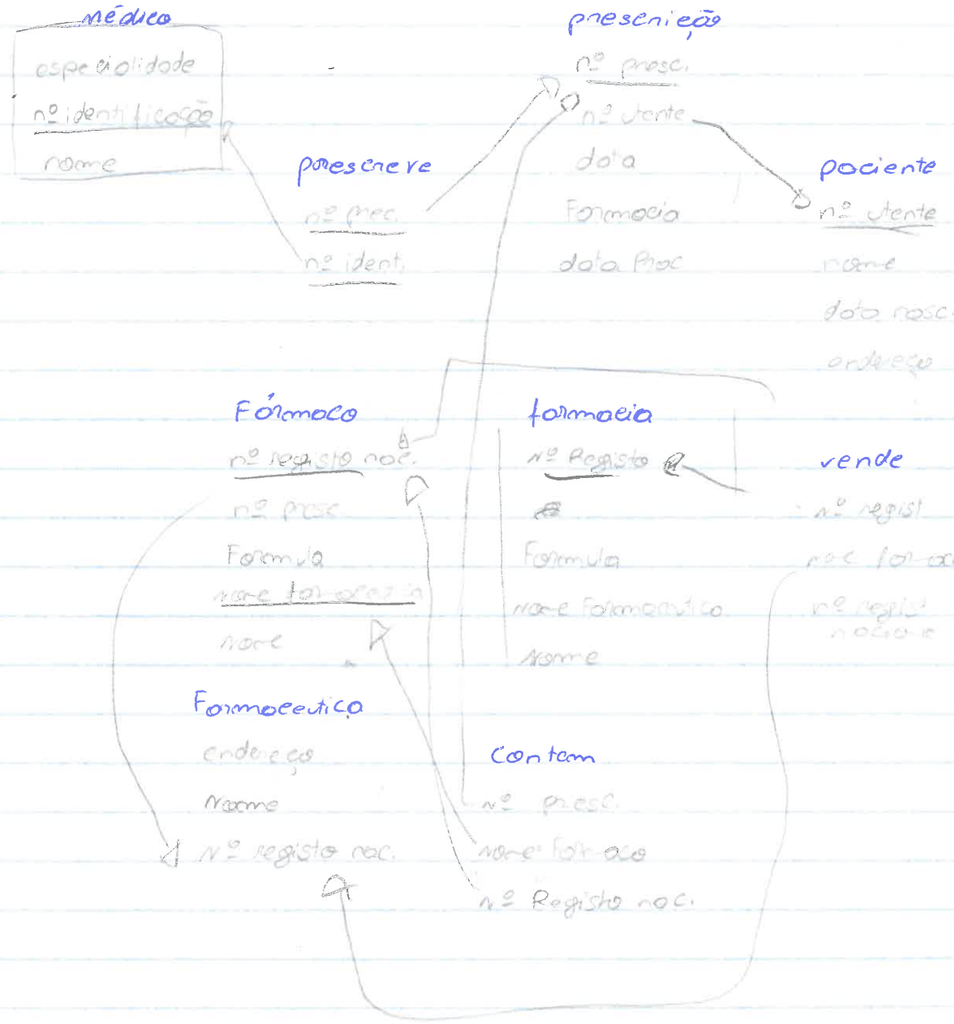


Porto LX 1880 | 18:30

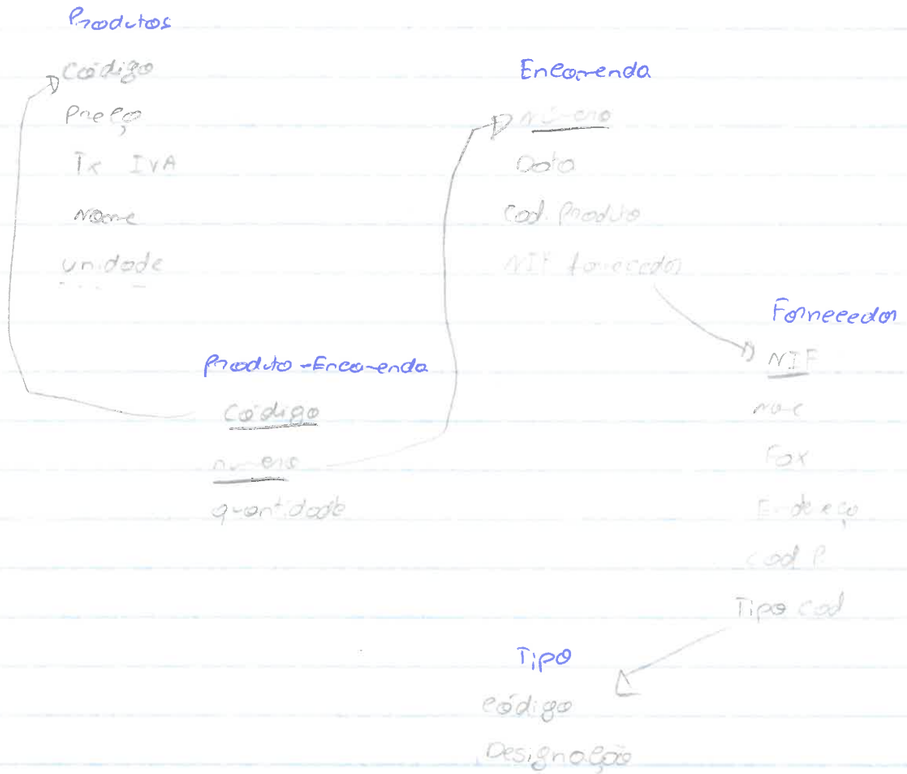
LX – REC 21:00 | 06:00+1

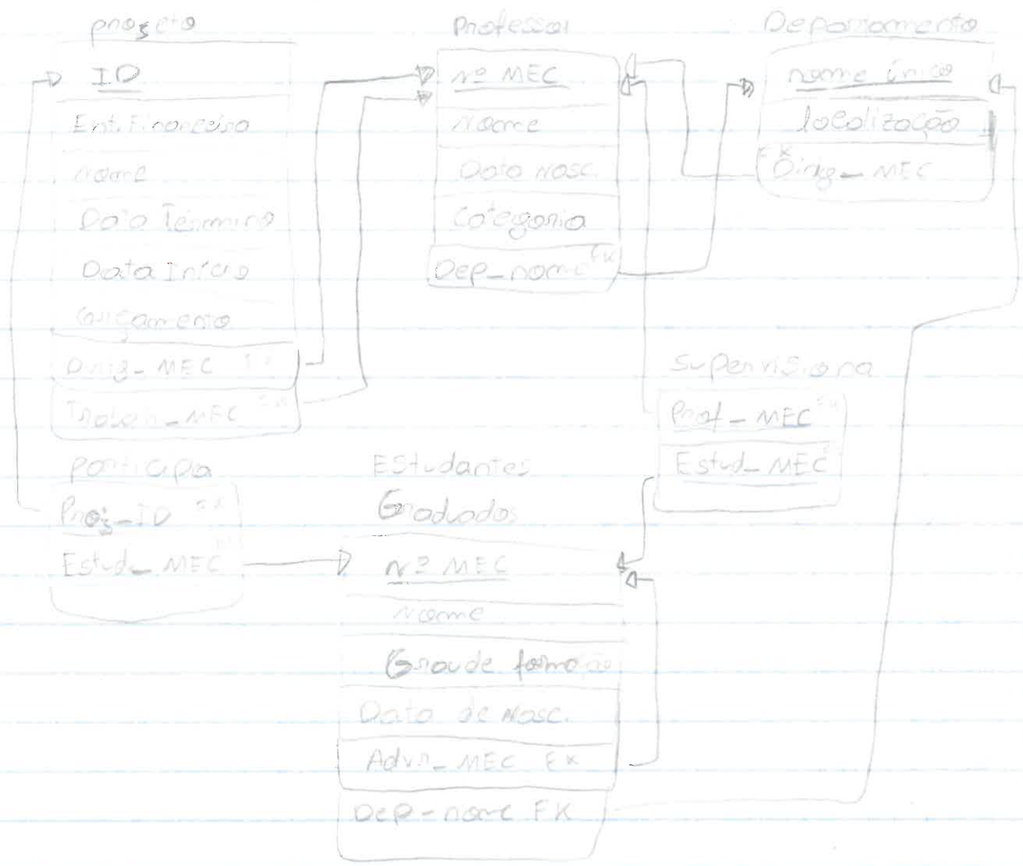
REC – NS 09:00 | 14:00

P2.2



Pz.1





Como passar o Modelo Relacional para SQL?

- ~~usando o sql server podemos~~

Managment Studio → editor de texto para SQL

Linguagem de SQL

2 sublinhagens principais:

DDT -> Data Definition Language

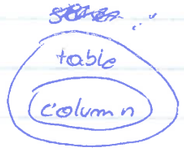
DML -> Data manipulation Language

1 sublinguagem de BD

↳ Controlo de base de dados

Objetos SQL

Há um hierarquia entre objetos



Cada instrução termina com ";"
Comentar -> "--"
Comentar bloco -> /* .... */

Data definition language -> permite definir vários entidade do BD

nota importante:
Sempre que se usa um SGBD deve-se consultar a sua documentação.

Criar Base de dados:
CREATE DATABASE Company;
DROP " "

CREATE SCHEMA

Tipos de dados
- Numbers
- Date and time
- Characters and Strings

Deveremos utilizar tipos de dados standard para aumentar a portabilidade do sistema

Objetos como imagens e outros ficheiros, não devem ser colocados na Base de Dados em si, cria vários problemas de back up.

char(n) -> cadeia de caracteres com tamanho fixo n (Ex: Matrícula sabemos sempre o tamanho)
varchar(n) -> " " " com tamanho máximo n
↳ Ex: Nome de uma pessoa

numeric (p,s) — p: precisão; s: parte decimal
ou
decimal(p,s)

Ex: numeric(2,4) -> 00,12

Domínio

Permite-nos definir tipos de dados, aplicando restrições

Ex: CREATE DOMAIN compsalary
NOT NULL CHECK (compsalary > 457)

SQL server
Há o type -> apenas permite verificar se é ou não null

CREATE TABLE EMPLOYEE (Fname varchar(15), Ssn INT
atributo — Domínio
Ainda podemos aplicar restrições aos atributos a atributos após a separação por vírgulas.

Tal como em java tem haver consistência no escrito, Ex:
As class de nome dos classes são em letra maiúscula.

UNIQUE → O atributo não é repetido mas pode ser NULL

Uma das maneiras de garantir a representar uma relação obrigatória é acrescentar NOT NULL

Se Eliminarmos a Entidade cuja chave primária é estrangeira noutra entidade o motor não irá deixar pois irá violar a integridade da Base de Dados.

Assim podemos preparar as entidades em caso dessas ocorrências

"on delet" → podemos definir que operações ocorrem em caso desta operação.

Álgebra Relacional

Seleção → ato de selecionar uma linha usando uma condição boleana.

Denotada por σ (sigma)

Ex:

σ Dno = 3 (employee)
↳ trabalhadores do depart. 3

σ Dno = 3 AND Salary > 1400 (employee)

troba. no depart. 3 e com salário superior a 1400

Em SQL:

SELECT * FROM EMPLOYEE
WHERE Dno = 3

Projeção → selecionar colunas

π Lname, Fname (σ Dno = 3)

Das linhas selecionadas apenas projeta o Lastname e o Firstname

SQL:

SELECT DISTINCT LName, Fname WHERE ...

O distinct serve para mostrar os valores repetidos

Também possível renomear

União - A união de 2 tabelas requer que as tabelas tenham o mesmo tipo de atributos bem como o mesmo nº



duplicados são eliminados

UNION (ALL)
↳ não elimina dupli.

Intersegção



Diferença



Produto cartesiano

Todos os combinações possíveis entre dois conjuntos

Ter atenção: pode originar inversais como entradas na tabela

outer join - os tuplos que não se emparelham também é selecionados, mas como não emparelha com o da outra tabela na tabela em não emparelham mete a null

Pode ser util para saber quais os tuplos que não participam na emparelhagem

Agregação

SSn		Salary	Dno
		400	1
		700	2
		1400	1
		1300	3
		2000	3
		500	3

Fat a média de cada Departamento

Dno ℱ avg(salary)

Dno	AVG (salary)
1	—
2	—
3	—

Problema 5.1

a)

(work_

b)

$\sigma_{Lname='Gomes' \wedge Fname='Carlos'}$ (employee)

c)

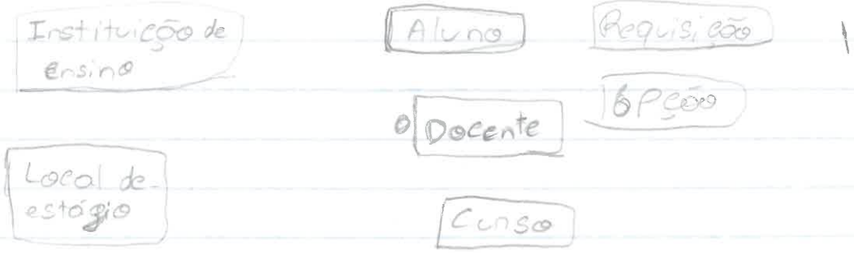
employee ⋈ count(nums) (work_on)

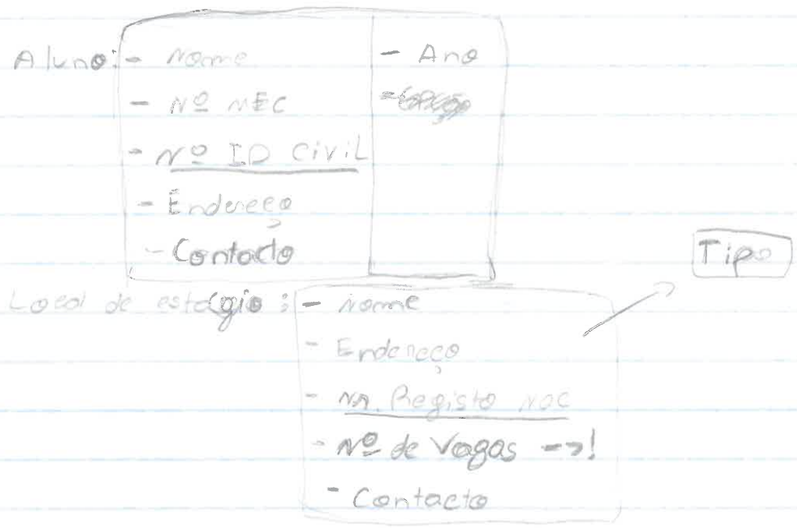
d)

Idastibo.

Projeto Locais de estágio

Entidades:





Curso: - Nº ID
- Nome

Docente: - Nome
- Nº ID civil
- Endereço -> ?
- Contacto

Instituição de ensino: - Nome
- Endereço
- Nº Registo
- Contacto

Linguagem SQL - DML

A partir da aula passada vamos mapear o conceitos para SQL

INSERT INTO "tablename" ("nome", "ID", "Data nasc"

se não temos os valores todos

Podemos especifica os atributos

INSERT INTO "tablename" (Dno, Nome, ..,)

(4, Jorge, ...)

DELETE FROM "tablename" WHERE "Atributo" = X

também aporto condições lógicos

Operações em conjuntos

SELECT * FROM "Tablename";

todos os tuplos onde, mesmo que sejam repetidos

" ALL Salary FROM EMPLOYEE

" DISTINCT " " ( )

↳ Não há repetições

Renomeação é uma operação importante para eliminar a ambiguidade

SELECT Fname + ' ' + LName AS Name

Produto cartesiano

SELECT * FROM EMPLOYE, DEPARTMENT;
↳ Colocamos restrição

Junção com Ambiguidade e Renomeação

SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn

Neste caso temos de eliminar a ambiguidade

Tratamento de NULL

NULL -> valor desconhecido ou não existe

As funções de agregação normalmente ignoram o NULL, para verificar usa-se o ISNULL

Em SQL as comparações lógicas com o valor NULL resultam em UNKNOWN

Nova regra do JOIN

... FROM EMPLOYEE JOIN DEPARTMENT ON Dno=Dnumber
WHERE Fname = 'Ricardo';

Agregação

Dno É medsal = avg(salary)

SELECT
FROM
WHERE
Group By

COUNT(*) → Nº de linhas

COUNT("Atributo") → Conta o nº de vezes que aparece o atributo sem estar a NULL

Após a agregação, se desejarmos fazer uma filtragem do resultado da agregação, não pode ser com o WHERE

Há a possibilidade correlacionar queries em que o sql pega num tuplo, faz o come o subquery e retorna algum valor desejado.

SQL - View

Apresentação de dados aos utilizadores, devem respeitar regras de privacidade.

Executam queries que podem ser mais ou menos complexas.

Há utilizadores que usam views de maneira excessiva, (tal como uma Maeno)

Perguntas sobre o join no agregação

```
SELECT Pname, sum(hours)
FROM (Employees Projects) ——> View virtual
GROUP BY Pname
```

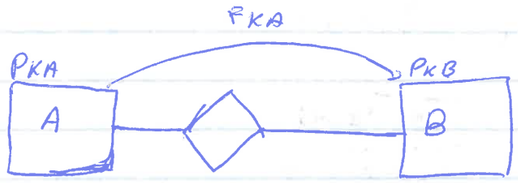
Trata como subquery, ao executar substitui o código, isto faz com que o desempenho não seja afetado devido ao parsing

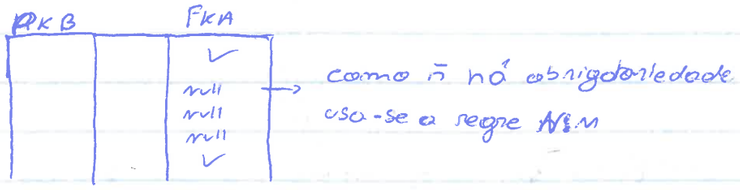
Podemos fazer inserts sobre as views mas temos de ter atenção em quais queremos fazer insert, caso a query que vá selecione chaves ou NOT null vá podemos fazer

Atenção sobre esta situação

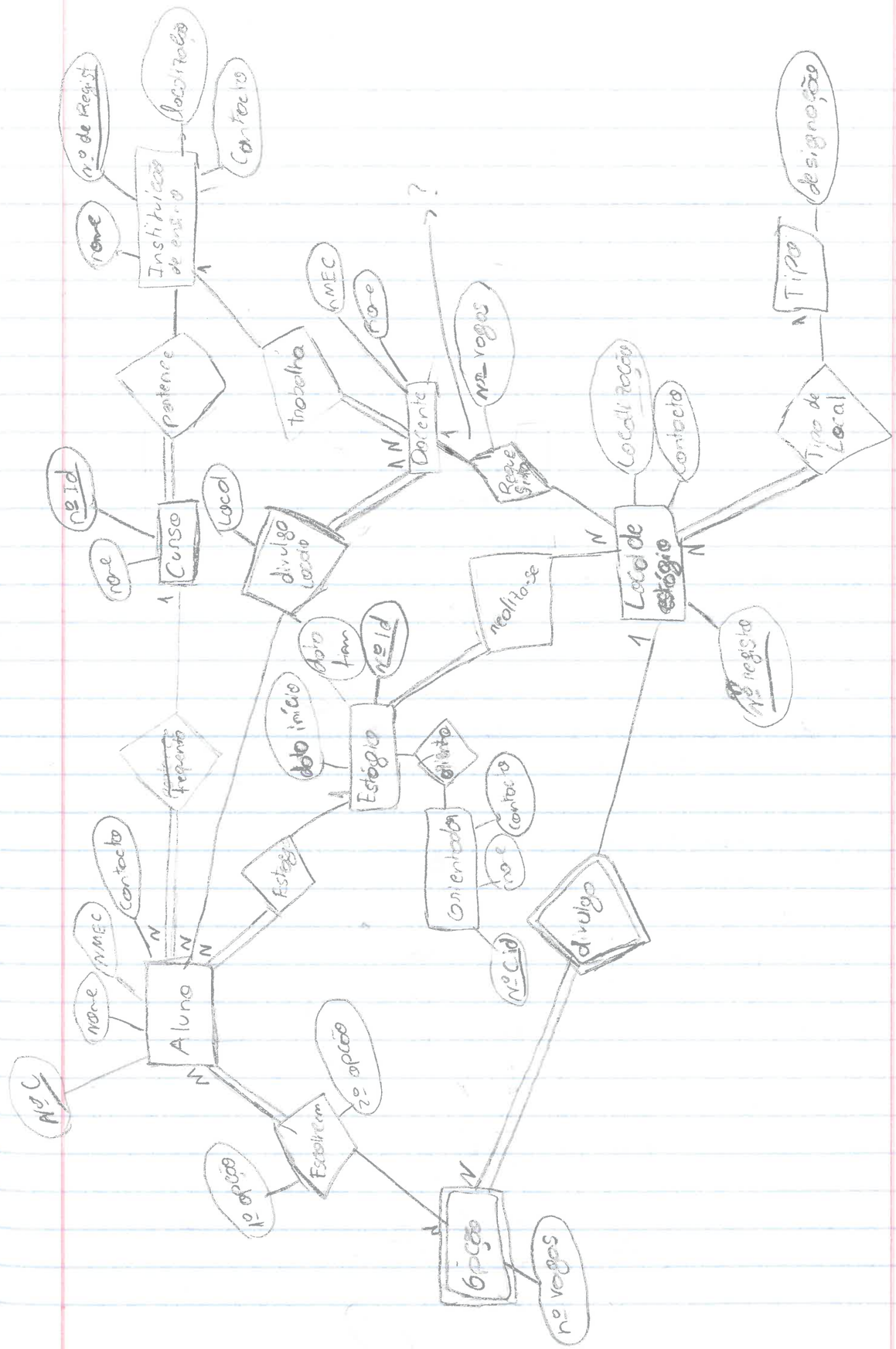
Devemos usar a opção WITH CHECK OPTION para que ele verifique os condições das vistas dessa maneira irá avisar se as operações fazem sentido.

Normalização

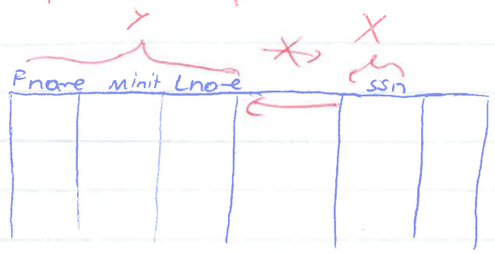




como vá há obrigatoriedade usa-se a regra N:M



Dependências funcionais



para aquele Ssn temos associados apenas aqueles nomes

Teste:

Construção de uma base de dados:

1º passo - Análise de Requisitos (texto)
- Levantamento de informação
- Filtragem de ruído

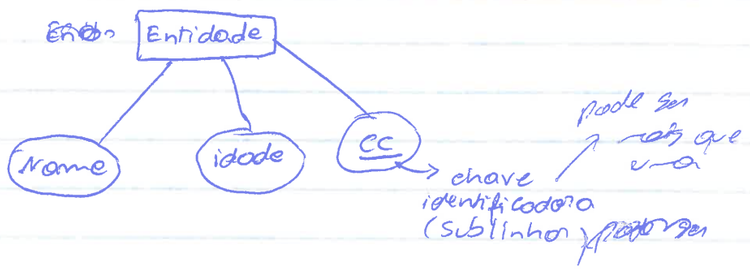
2º Desenho conceptual
- modelação e tratamento da entidades
- mapeia os entidades e relações do mundo real
- uma visão abstrata da estrutura da base de dados

3º modelo ~~Relacional~~ Entidade/Relação
- ~~Entidades~~ - algo que existe

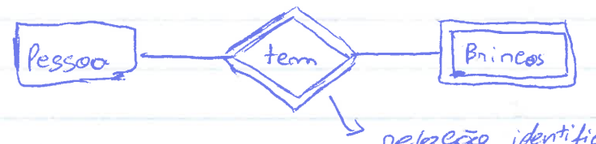
Entidade → Rectângulo

Disciplina

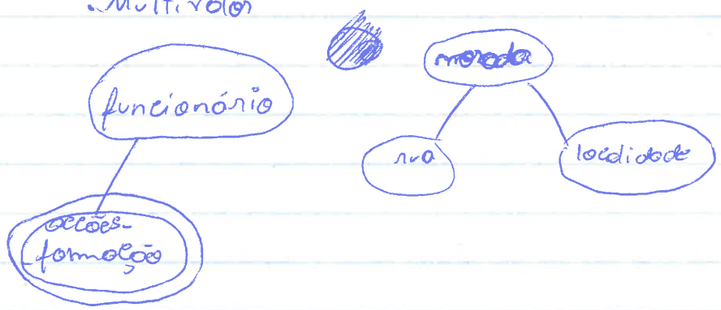
Atributo



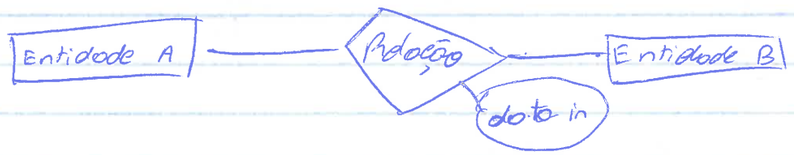
Entidades
- Fortes - não dependem de outros para serem identificadas
- Fracos - Dependem de outros para serem identificados



- Derivados → muda ao longo do tempo
- Multivalor



Relações - entre duas ou mais entidades



também podem ter atributos

Relacionamentos
- Grau - nº entidades na relação
  - ~~unidade~~ unária
  - binária
  - Ternária

Obrigatoriedade

Cardinalidade - nº de ocorrências

- Relações recursivas

Simétrica - ñ há distinção

assimétrica - há distinção entre papéis



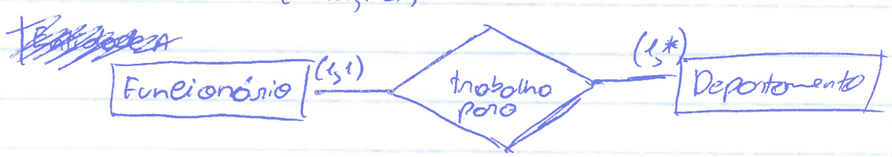
Obrigatoriedade

Linha dupla - a entidade participa em ~~todas~~ os na relação

" singular - a entidade ~~é~~ pode ou ñ participar

notação alternativa:

(min, max)



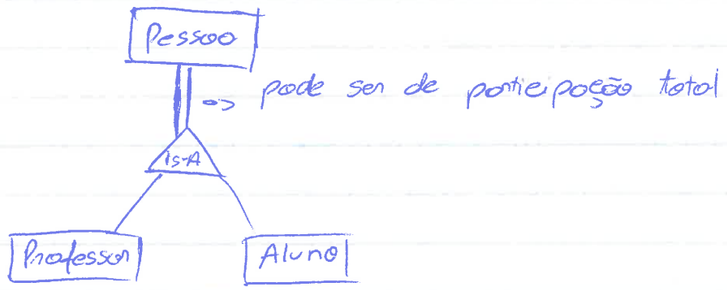
min = "0" se ñ for obrigatório
= "1" se é obrigatório

Um atributo pode generalizar ou ser especializado



Disjuntos: Entidade só pode pertencer a uma especialização

Sobreposta - pode ter mais do que uma especialização



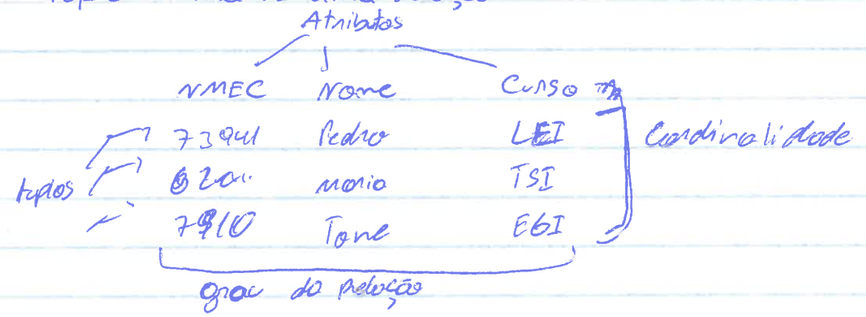
pode ser de participação total

- Modelo Relacional
  - Modelo baseado na teoria dos conjuntos
  - Representado por tabelas

Esquema da Relação
- nome e atributos

Pessoa (nome, bi, idade)

Tuplo = linha de uma relação



Esquema da relação - Estudante (nMec, Nome, Curso)

Chaves

superchaves - conjunto de atributos que identificam de forma única os tuplos

chave candidata - Subconjunto das superchave que ñ pode ser reduzido sem perder a classificação de chave

chave primária - atributo ou atributos selecionados de entre os candidatos para a

" única - chave candidata ñ selecionada para primária

chave Estrangeira - chave importada de outra relação

No escolha da chave primária temos de ter atenção ao facto de essa chave pode ser alterada à posteriori, aqueles que ñ foram escolhidos tornam-se chaves únicas

Integridade

Deve ser garatida pelo SGBD

- Domínio - conjunto dos elementos
  - Os campos devem obedecer às restrições impostas. chave primária
    Ex: NOT NULL
    Sexo char CHECK(Sexo='M' OR sexo='F')
- Entidade - todo tuplo deve ser identificado de forma única
- Referencial - A chave estrangeira ou é null ou então uma chave primária de outra tabela

Regras de codd

verificam se efetivamente uma base de dados pode ser chamada de relacional ou ñ. É um conjunto de 12 regras, o próprio codd admite que é difícil cumpri-las todas.

Conversão do modelo DER em modelo relacional

passo 1:
para cada entidade forte cria uma tabela com os seu atributos.

passo 2:
Para cada entidade fraca importar como chave estrangeira a chave primária de entidade dominante, assim a chave primária irá ser a combinação da chave estrangeira e a do própria fraca.

passo 3:
Em relações 1:1 escolhemos uma tabela em que a sua participação seja total e importamos a chave primária da entidade com que se está relacionada, quaisquer atributos da relação ñ também importados.
Em caso de ñ existir um dos lados com participação total trata-se como uma relação de N:M.

passo 4:
Para relações 1:N importa-se como chave estrangeira, na tabela do lado com N, a chave primária da entidade do lado com 1. Quaisquer atributos do relacionamento também ñ importada para essa tabela.

passo 5:

Para as relações N:M criar uma nova tabela como chave primária as chaves das entidades adjacentes.

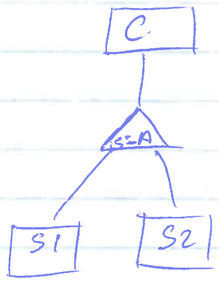
passo 6:

Atributos devem fazer parte de uma tabela com a designação do atributo e como chave estrangeira a chave primária da entidade na qual é atributos. A sua chave primária irá ser a combinação da FK e da designação do atributo.

passo 7:

Relacionamento n-ário (com n > 2), deve-se criar uma tabela nova importando as PK das tabelas adjacentes.

Em caso de especialização



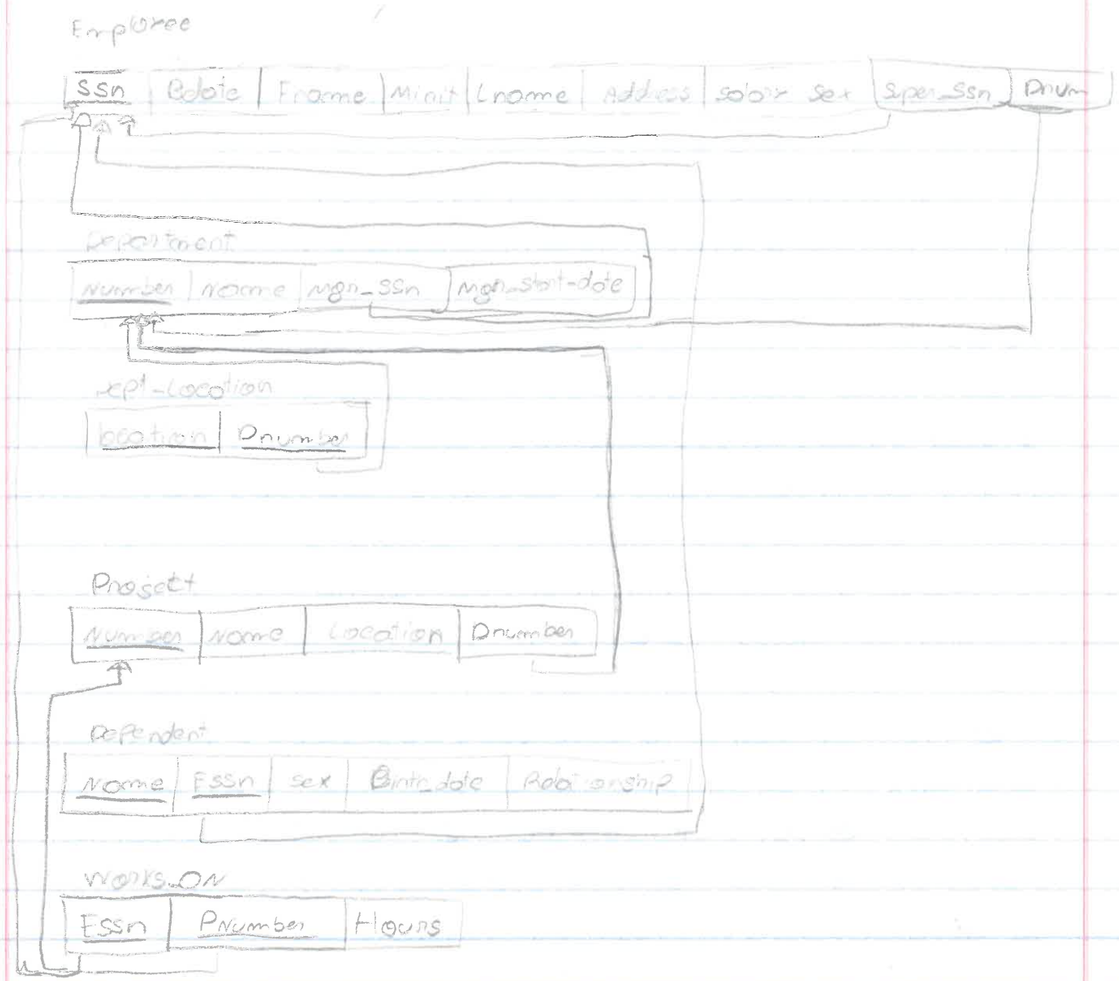
Pode-se fazer de duas maneiras:

1º Cria-se uma tabela para a superclasse e outras para os derivados que importam com chave estrangeira a chave da superclasse

funciona para casos de especialização total e parcial

2º Criam tabelas para cada uma das classes derivadas. Incluindo os seus atributos e os da superclasse.

Apenas funciona para especialização total.



Linguagem SQL

Divide-se em ~~dois~~ duas partes

DDL - Data definition Language

DML - Data manipulation ''

DDL

- CREATE DATABASE "dataBaseNome"
  DROP DATABASE ''

agrupamento de tabelas
↓
CREATE SCHEMA "schemaNome"

Domínio

CREATE DOMAIN CompSalario INTEGER
NOT NULL CHECK(CompSalario > 449)

↑
Não existe em sql

CREATE Type SSn FROM Integer NOT null
CHECK (SSn > 0);

Alter table "tablename" DROP COLUMN
" CONSTRAINT
ADD ''

ALTER TABLE "tablename" ALTER COLUMN "columnName" CHAR(9)

constantes de 9 caracteres

Algebra relacional

- União - ocorre entre duas tabelas com o mesmo nº de atributos e com o mesmo domínio.
  ∪ → Resulta todos os tuplos exeto duplicados

Intersecção - necessito dos mesmos condições que a união e resultam os tuplos iguais em ambas tabelas
∩

Diferença - mesmos condições da união mas devolve os tuplo que apenas se encontram fora da intersecção das duas tabelas. −

nomes dos funcionários que são gestores de departamentos

$\pi_{Fname, Minit, Lname}(EMPLOYEE \bowtie_{Ssn=Mgr_Ssn} DEPARTMENT)$

Junção

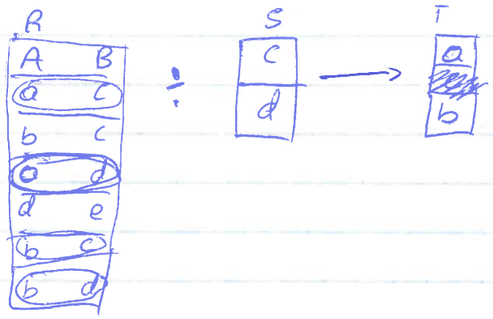
Equijoin

Natural join
Faz a junção dos colunas com o mesmo nome

Nota: Por vezes recolhe-se a renomeação para facilitar a junção.

Divisão

6 resultado incluirá todos os tupo que est relacionados com os atributos usados na comparação



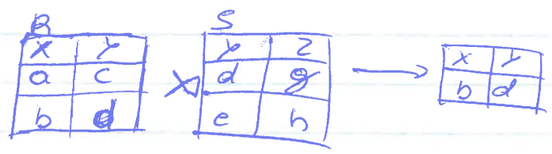
Inner join

Os tuplos sem correspondência são descartados

OUTER

Semi join

Left semi join



Faz a representação dos tuplos em junção natural dos tabelas R e S mas mostra os tuplos de R

Right join

6 comportamente é semelhante de cima mas faz a representação do tuplo do tabela do direito, S.

Inner join

6s casos anteriores s inner n representam os tuplos sem associações

Outer join

Faz a representação de todos os tuplos, mesmo aqueles sem associação.

Left Outer join

S representados tuplos da tabelas do esquerda juntamente com os associados, os sem associação S representados na tabela

Agregação

sum: soma dos valores

count: contagem dos valores

$\sigma_{farmaco=null}$ (FARMACO ⟕ codigo=farmaco Prescreve)

$num_consulta\,\mathcal{G}\,count^{*}(farmaco)$ (Consulta ⟕ num_consulta=consulta Prescreve)

$tmp \leftarrow \Pi_{medico,\,consulta,\,num_farm=count(farmaco)}$(Prescreve)

$\Pi_{medico,\,medio_farm_consulta=avg(num_farm)}$(tmp)

SQL DML

INSERT INTO "tablename" VALUES (...)

↑ inserir valores numa tabela

DELETE FROM "tablename" WHERE "Alguma condição"

↑ Eliminação de um tuplo

Sem WHERE, elimina-se todos os tuplos

UPDATE PROJECT
SET Plocation = 'Bellaire', Dno = 5
WHERE Pnumber = 10;

↑ atualiza um tuplo

UPDATE EMPLOYEE
SET Salary = Salary * 1.1
WHERE Dno = 5;

↑ afeta n-tuplos

ALL -> seleciona todos os tuplos, mesmos os duplicados
DINSTINCT -> Apenas ~~os~~ ~~os~~ ~~sem~~ Sem repetições

∪ - R1 UNION R2
∩ - R1 INTERSECT R2
- - R1 EXCEPT R2
↑ diferença

Produto cartesiano
SELECT * FROM "tablename1", "tablename2";

Comparação de Strings

SELECT *
FROM PROJECT
WHERE PLOCATION LIKE '% Houston, Texas %';

↑ zero ou mais caracteres

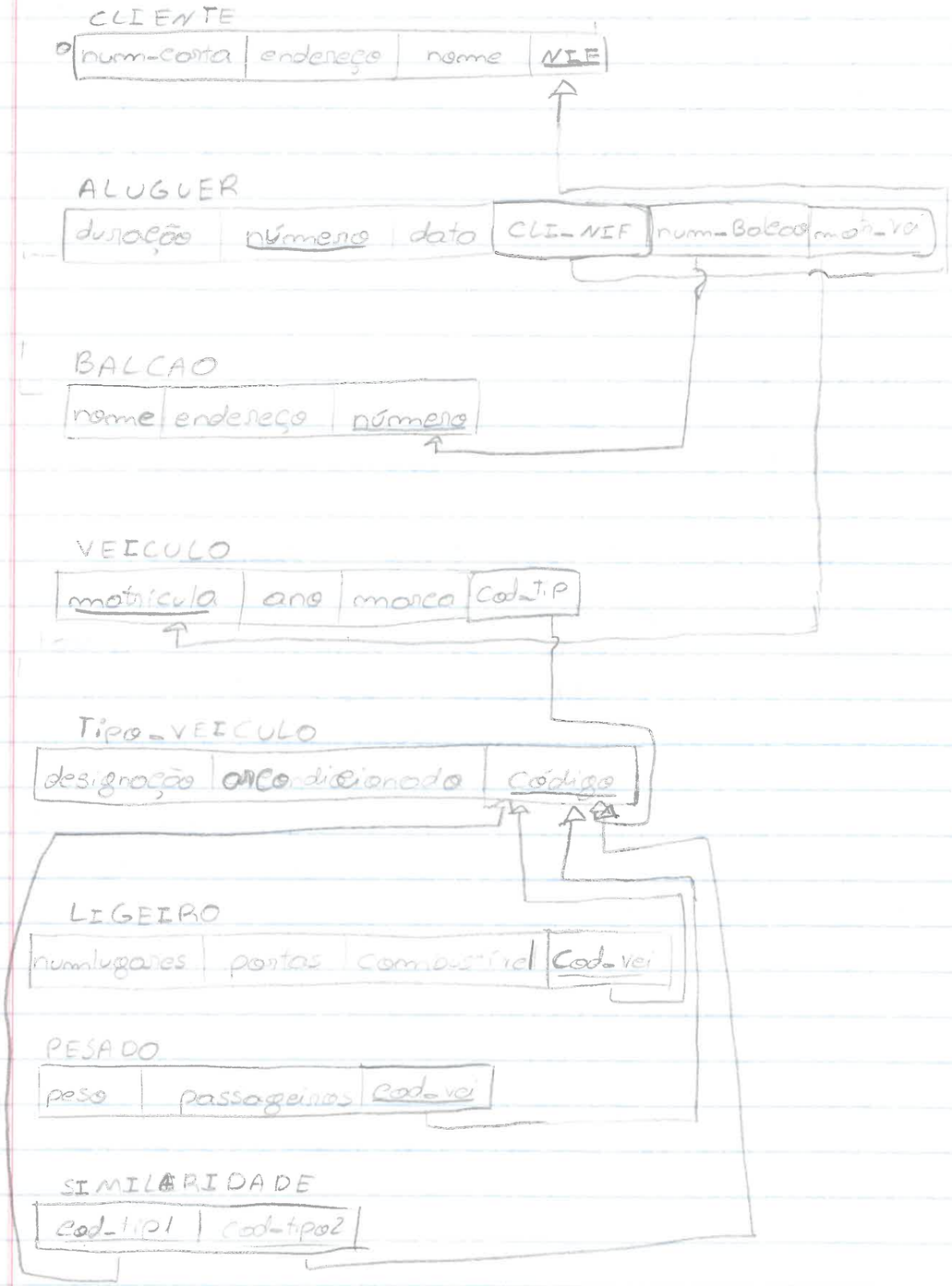
caracter de comparação
.... LIKE '_ _ 5 _ _ 6 _ _ _';
↑ um qualquer caracter

WHERE (Salary BETWEEN 3000 AND 4000)

Teste do ano passado

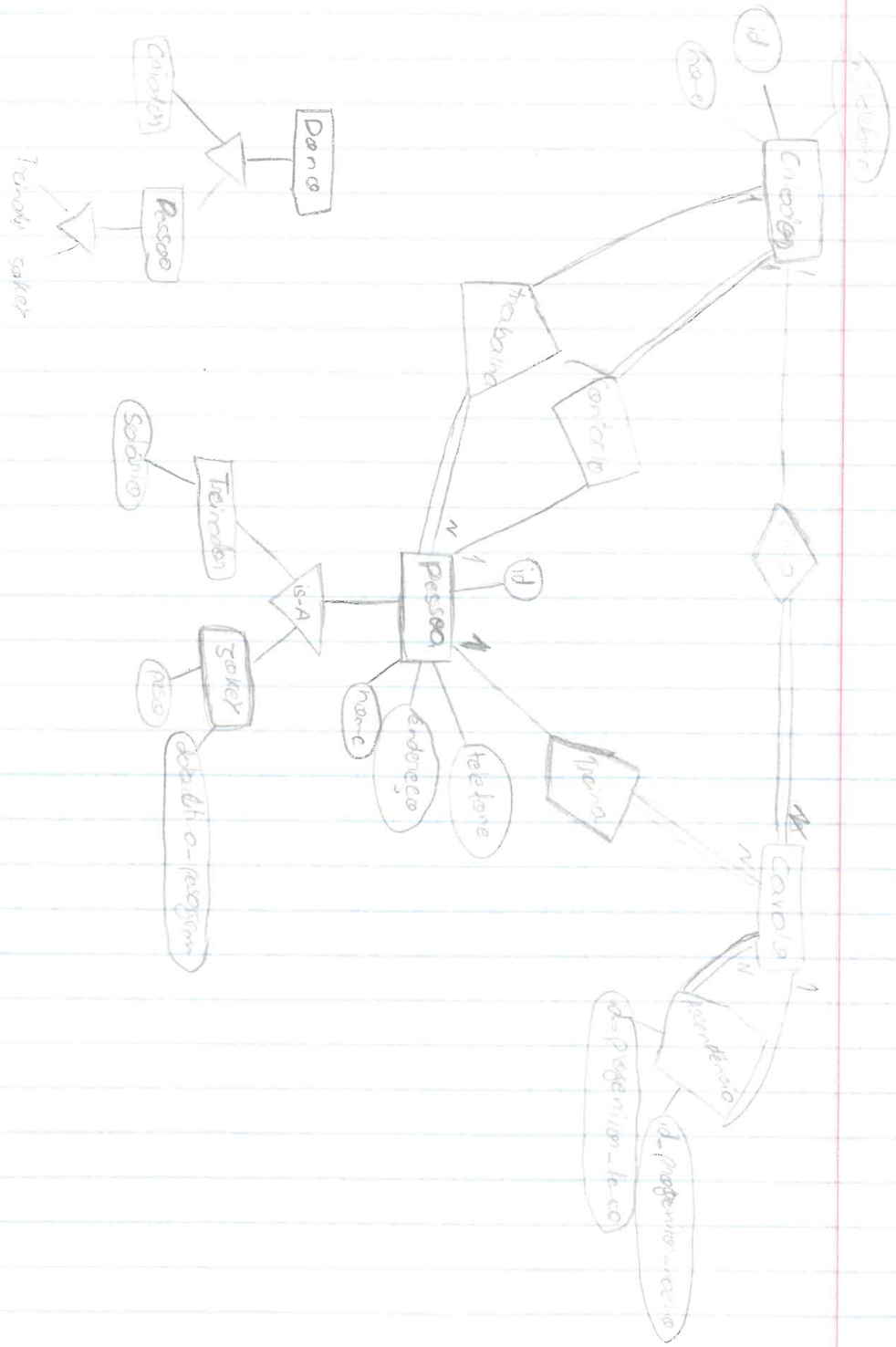
Q1 a) V
b) F
c) V
d) V
e) F
f) V
g) F
h) V

Q2 a)


CLIENTE
num-conta
endereço
nome
NIF
ALUGUER
duração
número
data
CLI-NIF
num-Balcao
matrícula
BALCAO
nome
endereço
número
VEICULO
matrícula
ano
marca
CodTip
Tipo_VEICULO
designação
ar condicionado
Código
LIGEIRO
numlugares
pontas
Combustível
Cod_vei
PESADO
peso
passageiros
cod_vei
SIMILARIDADE
cod_tipo1
cod_tipo2


critico
N
tem
M
especie_tede


tem
id espec
id critico



ViewW

- Relação virtual
- São representações dos dados de tabelas, ñ há segundo armazenamento de dados
- É possível manipular dados das views
- São utilizados em adaptações de esquemas de bases de dados a diferentes aplicações
- Também são usados para tratar de questões de segurança e integridade

```
CREATE VIEW "some_name" AS
    SELET name, Fname
    FROM PEOPLE
    WHERE (Salary BETWEEN 3000 AND 4000);
```

Após criação de uma view é possível efectuar operações sobre ela.

Nested view

- views que usam outras views como fonte

É possível inserir dados na view, com o insert into mas por vezes não fazem sentido, quebrando assim a integridade de dados, para evitar essas situações deremos colocar o comando WITH CHECK OPTION.

```
ou ALTER VIEW "name" AS
CREATE VIEW "name" AS
    SELECT Fname, Dno
    FROM EMPLOYEE
    WHERE Dno = 5
    WITH CHECK OPTION;
```

Neste caso, se tentarem fazer

```
INSERT INTO "name"
VALUES ('sose', 8)
```

erro

Verificação ↙

Projeto queries:

Listar opções para o Alunos

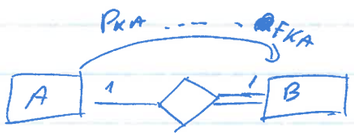
Listar locais de estágios para o Docente → Pessoa - either

Escolhas por Aluno

Análise de Qualidade

Critérios informais: - Redução de null

...



pq ñ proceder ao contrário?

para reduzir os NULLS, Se A ñ se relaciona-se com B ou um campo inicia estará a NULL

~~Se caso~~

num caso de uma rel. se obrigatoriedade então-se uma tabela nova que ñ as ináp possuin NULL, se existe a relação, existe tuplo

Dependências Funcionais:

~~Pode ser~~ X → Y

X define Y ou Y depende de X

Ex:

ssn → {Fname, Lname, Salary, ...}

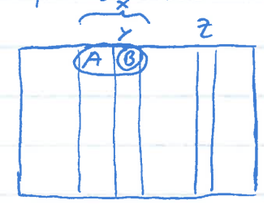
ssn define de forma única ↑

dnumber → dName

sempre que um centros dNumber aparece, tem que estar associado a só nome

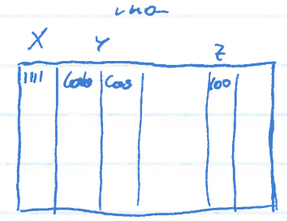
Axiomas de Armstrong

- Reflexibilidade: $Y \subseteq X \Rightarrow X \rightarrow Y$



- Aumento X → então XZ → YZ

ssn Fnome



X → Y

XZ → YZ

- Transitividade

X → Y e Y → Z ⇒ X → Z

- Dependência total

Atributo depende do chave

- Dependência parcial

Atributo depende de uma parte da chave

muito mais importante ver projeto

Primeira Forma normal (1NF)

Um atributo apenas deve conter um campo, ou seja, um nome deve ser dividido num atributo Fname e Lname ou morada em rua, nº porta, Freguesia

Segunda Forma normal

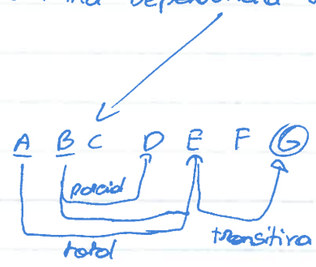
Eliminar dependências parciais

ver exemplo da aula

Numa tabela queremos eliminar a redundância, se na tabela tivermos o ssn de funcion. e Pnumber de projetos. Funcionário que trabalhem no mesmo proj. iria fazer aparecer o Pnumber n vezes.

Terceira Forma normal

Elimina dependência transitiva



Há mais formas normas normais, até à terceira é só vantagens a partir dessa para à frente é necessário ter cuidado pois só começam a acrescentar-se desvantagens.

BCNF -> Boice-Codd Normal Form

↳ Ainda não percebido -> ver exemplo

A 4NF e a 5NF são raras

4NF Dependências Multivalor

X	Y	Z
x_1	y_1	z_1
x_1	y_1	z_2
x_1	y_2	z_1
x_1	y_2	z_2

-> Podemos decompor

Pode-se usar algoritmos para detetar

5NF Dependências de junção -> Ainda mais rara

Pode-se decompor as tabelas e por dependência de junção é possível economizar algum espaço.

Fazendo a junção das tabelas de maneira a obtermos a original estamos na quinta forma normal.

Indexação e optimização

É mais eficiente fazer uma seleção/projeção de dados sucessivos e de seguida fazer as junções do que juntar tudo numa tabela e de seguida fazer essa seleção/projeção.

```
SELECT Fname, Lname, Patient_ID
FROM Patients
WHERE Fname='Maria' AND Lname='Pinto'
```

O SGBD abre todas as tuplas e procura 1 a 1? -> O(n) de complexidade

Isto não é executável pelo SGBD e usa estruturas de Dados

Índices -> estruturas de dados que permitem pesquisas, têm um ponteiro para a localização física

- Não Denso: Início da página
- Denso: Offset para a localização do tuplo

Vantagem -> Mais eficiência na pesquisa

Desvantagens -> Também ocupam espaço e na inserção de dados tem que haver uma inserção na tabela e uma atualização.

Nos parametros dos indices podemos estipular o nivel de complexidade de procura ex: procura por nome Mario e procura por Mario Pinto

Single-level → Armazenam o valor do atributo indexado e a sua localização

Primary index* → Indexa atributo chave da relação

Cluster index* → Indexa atributo que não é chave, resulta normalmente em varios tuplos, ou seja, é não denso, ná varios tuplos a serem referênciados por esse index.

Secondary index* → valor duplicados que seja necessário indexar de maneira mais especifica exista-se um índice para um indice que aponta para o tuplo

8.1

a) Os atributos são indivisíveis

b)

Cima

B | A | D |

R(A, B, C, D, E, F, G, H, I, J)

2FN (R1(A, B, D, E, G, I)
R2(B, C, D, E, F, G, H))

3FN (R(A, B, D, F, G, I)
R(D, E, F)
R(G, H)
R(B, C))

Cima
Scannable

8.2

a)

R = { A, B, C, D, E, F, G, H, I, J, S }

A chave é o atributo

A e B

b)

2FN $\begin{cases} R_1(\underline{A}, \underline{B}, C) \\ R_2(\underline{A}, D, E) \\ R_3(\underline{B}, F, G, H, D, I, J) \end{cases}$

c)

3FN $\begin{cases} R_1(\underline{A}, \underline{B}, C) \\ R_2(\underline{A}, D, E) \\ R_3(\underline{B}, F) \\ R_4(\underline{F}, G, H) \\ R_5(\underline{D}, I, J) \end{cases}$

8.3

a) $R = \{A, B, C, D, E\}$

A chave são os atributos A e B.

b)

2FN — $R(\underline{A}, \underline{B}, C, D, E)$

3FN $\begin{cases} R_1(\underline{A}, \underline{B}, C, D) \\ R_2(\underline{D}, E) \end{cases}$

c)

BCFN $\begin{cases} R_1(\underline{A}, \underline{B}, C, D) \\ R_2(\underline{D}, E) \\ R_3(A, \underline{C}) \end{cases}$

8.4

a) $R = \{A, B, C, D, E\}$

A chave são os atributos A e B.

b)

c) 2 FN { $R_1(\underline{A}, \underline{B}, D, E)$

d)

3 FN { $R_2(\underline{A}, C)$

BCFN

8.5

a)

Pessoa(ncid, contacto, nome)

Aluno(n_cid, ano_curricular, nomec, idcurso, nome_curso)

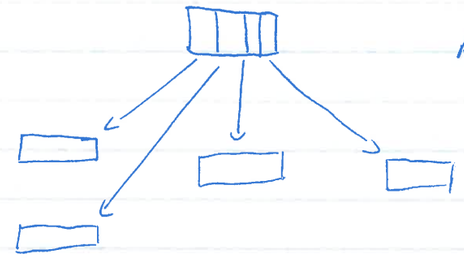
{n_cid} ——> {ano, ...}

Indices - São muito importantes no ramo de base de dados, é os que têm melhor fonte de rendimento

Aceleram processo de pesquisa mas também ocupam espaço em disco

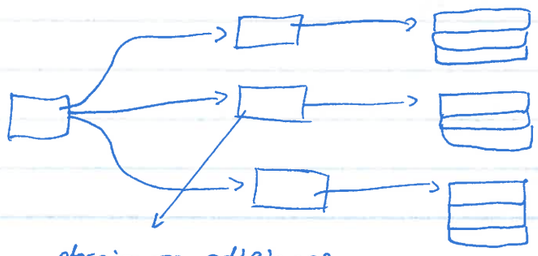
~~Secondary index~~ :

Multilevel Index



podemos ter vários níveis de indexação, tornando a busca mais eficiente pois a complexidade da busca é reduzida

Um problema associado com esta estrutura é uma nova indexação que tenha de ser feito no índice que já está cheio vai obrigar a fazer page split. Ou seja, as indexação têm de ser copiado do outros ni índices para allocar espaço



desejamos adicionar uma indexação se já estiver cheio teremos de alocar espaço

SQL - Index

CREATE INDEX idxPname ON Project(Pname);

podemos ter indices com mais do que um atributo

((" idxEmployee ON Employee(Fname, Minit, Lname);

Ao desenvolver interfaces temos de ter consciência do que da implicações que isso trás para a base dados

↳ "Este conhecimento vale dinheiro"

Numa experiência:

total de registos

	Records Returned	Cost	I/O Reads	
①	72 591	.485	526	→ Leituras de todos
②	9	.003	2	→ Leitura com índices
③	11 05	.004	4	

2 leituras podem originar de dois níveis de índices



Deve-se evitar operações de I/O, um bom índice evita isso

Moderação da indexação:

Temos de evitar índices que têm valores repetidos

Ex: pesquisa de uma pessoa com índice que aponta para o Concelho onde ela vive. Iria resultar em muitos tuplos

A escolha do índice deve atender a alguns compromissos! ~~Completar~~ ver slide 18 Índices

Há situações em que não sabemos em que contexto foram desenvolvidos certas queries, índices, etc.

se pretendermos ter uma ideia de como é usada podemos usar ferramentas para verificar o histórico.

Temos de garantir que a base de dados funcione a partir do momento zero temos de garantir que a base de dados está pronta para suportar dados em grande quantidade, e eventualmente temos de preparar as nossas aplicações para qualquer eventualidade que possa ocorrer durante a sua utilização a ~~priori~~ posteriori em vez de pensar só no momento de criação

SQL Server - Indexing

Tipos de índice: • Clustered → A estrutura de dados esta associada ao nó de pesquisa

Neste caso só possível ter um índice clustered, ~~não~~ tem ponteiro para outra estrutura, ver slides.

~~mas podemos ter vários índices clustered~~

Ver slides

• Non-clustered → • A indexação não é feita pela chave primária mas por outros atributos até chegar à chave primária e depois é possível ir à ~~chave primária~~ estrutura principal para para buscar a informação.

Page split

Em clustered index ñ tem problemas de page split, se a chave pk for um nº ou algo sequêncial.
Ao contrários dos non clustereded.

~~Fazer para isso~~

Para evitar isso temos FILL FACTOR, INDEX PAD
↳ se queremos fazer FILL Factor nos tabelas intermédias

podemos de definir até que percentagem queremos encher as nossas ~~tabelas~~ reduzindo assim os page split mas desperdiçando espaço

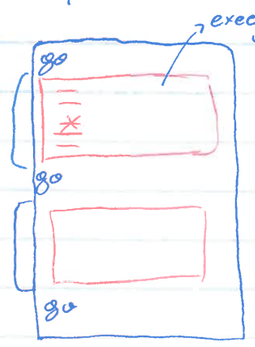
Programação em SQL

Pensando na tabela Empfare com várias dependências

Temos de pensar se no contexto dos nossos dados vale a pena eliminar o funcionário ou meter ativo ou ñ

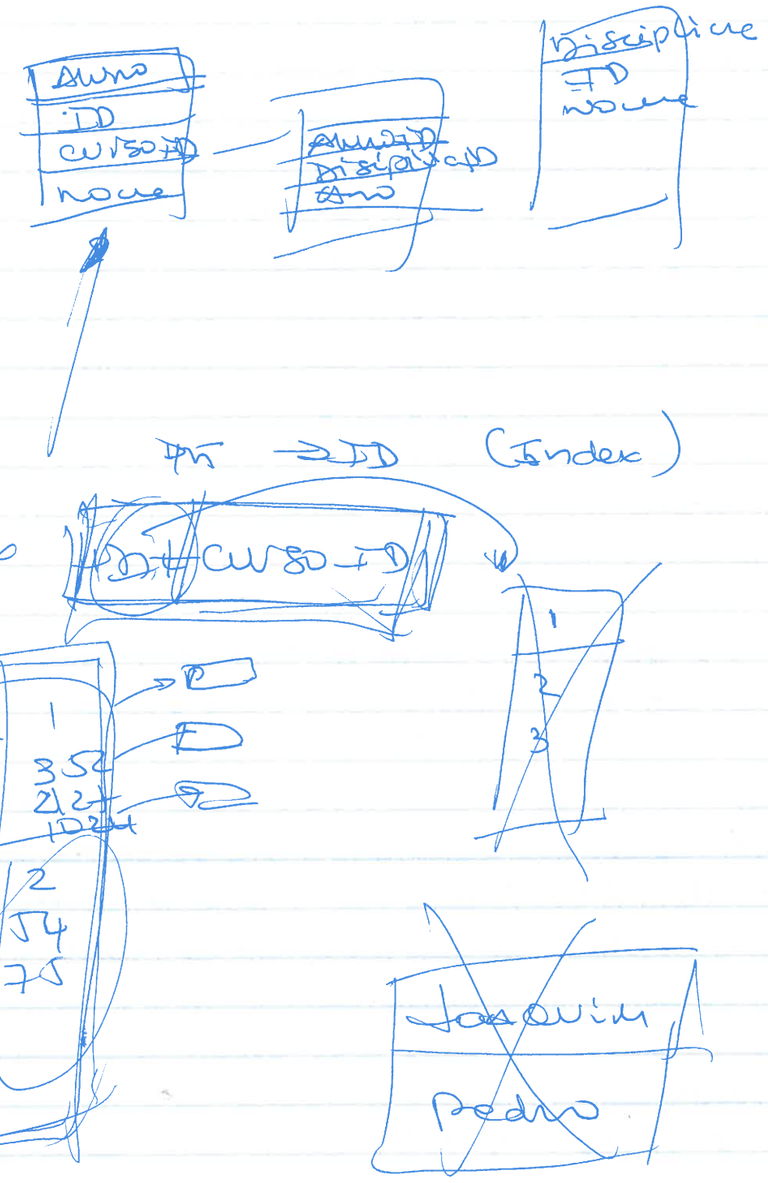
Para eliminar o Empfare temos de verificar todas as suas dependências e só se ñ as tiver é que pode ser eliminado

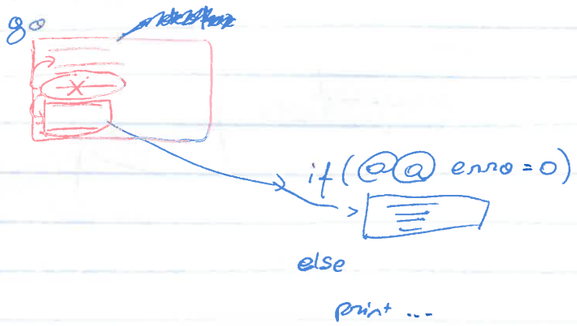
Batch → Grupo de instruções SQL que constituem uma unidade lógica



go → delimita uma batch

evita que, num nº elevado de instru., se uma tiver um erro, ele ñ pára devido a esse erro





~~Declaração~~

variáveis

Declare @x varchar(10) = '616'

" @dx as int
↳opcional

Atribuição de valores

SET @x = 'António';

SET @ y+=5;

podemos atribuir um valor logo no momento de programação

~~podemos~~ ctu

SELECT @price = price FROM Product

WHERE id = ...

NVARCHAR(...) -> codifica em unicode

Display de variáveis

SELECT @x

ou

Print 'var x: ' + str(@x);

cada Batch tem um scope, variáveis declaradas numa batch não são reconhecidas noutra

Nota:

Evitar key words como ORDER no nome da tabela

switch

CASE ... WHEN

Tabelas Temporárias

Permitem armazenar mais do que um valor

CREATE ?

DECLARE @Temp TABLE (PK INT PRIMARY KEY, ...)

INSERT INTO @Temp ...

Não podem ter associações de chaves ~~p~~ estrangeiras

Também está limitado pelo scope do batch se forem declaradas como variáveis

~~DECLARE #Hello (~~

o # afirma que a table não esta limitado pelo ~~scope~~ scope do batch

CREATE TABLE #Hello

Mas está limitado por sessões

Também podemos criar uma global, comum a outras sessões

CREATE TABLE ##Hello

~~está~~ está localizadas num diretório ~~d~~ do motor de BD

Cursors

São muito exigentes em termos computacionais.
São instruções executadas sequencialmente

Ex: usando uma query declara-se como cursor e executa-se o cursor, por cada vez que vai ser executado vai iterar sobre o cursor retornando os resultados após cada iteração

Têm que ser abertos, fechados, e a sua memória tem de ser libertada

Só devemos utilizar os cursores quando necessário ver situações recomendáveis

10.3

a) CREATE CLUSTERED INDEX ix_func ON EMPLOYEE

Procedimentos

Para eliminar um funcionário teríamos que apagar todas as instâncias de determinado funcionário na tabela WORKS_ON, Dependente, devido ao ugr_ssn e só depois de eliminar essas dependências é que poderemos eliminar um tuplo do funcionários

Todos estes procedimentos podem ser implementados num procedimento, assim numa simples operação DELETE faz todos estes procedimentos

É util ao trabalhar em equipas

Um Procedimento é mais eficiente pois fica armazenado em cache e apenas tem que ser executado, não sequer compilação

Ex de utilidade num dissoint podemos garantir a integridade de dados, criando um procedimento para adicionar um tuplo fazendo a verificação se ele existe na outra tabela.

Perceber os mais vários dos procedimentos

Podemos tratar isto como classes em programação aplicando as regras de programação por contrato. O cliente só precisa de mexer nos procedimentos e não deve ter manipulação direta

Os procedimentos não estão associados a uma tabela

Funções

Qual a diferença entre funções e procedimento?
Têm as mesmas vantagens em termos de performance do que os procedimentos.

Num podemos usar SP como fontes de dados

exec mySP...
select * from mySP com as funções é possível fazer isso.

Existem 3 tipos de funções:

Escalares: Retornam escalares
Ex: variáveis do tipo money, int, varchar

Úteis em CHECKS com uma lógica complexa

Em vez de usarmos views podemos criar Views parametrizáveis
Assim temos uma maior capacidade operacional

Triggers

São procedimentos que são ativados quando ocorre determinada operação

Ex: para assegurar um disjoint nas tabelas, podemos usar um triggers que faça o tratamento dessa situação.

Podem estar associados a DDL e DML
só vamos tratar

Numa operação de inserção, é feito um conjunto de operações de verificação antes de fazer as operações

Triggers — ~~INSTANCE~~ INSTANCE OF

AFTER -> é possível fazer rollback

Os triggers têm acesso às tabelas temporárias inserted, deleted estas tabelas para fazer operações sobre elas.

~~Ao intersetarmos uma~~
Num trigger se chegarmos à conclusão que afinal é para continuar chamamos outra vez a operação mas na segunda ação o trigger não é acionado.

~~Sábio~~ verificar o sábio mensal para pôr a projeto

tabela e player - tem orçamento
- ver os funcionários no projeto e os respetivos salários

Transações

Conjunto de instruções atómico, ou são todas executadas, ou nenhuma é

Há operações que não podem ser interrompidos a meio se tal acontecer deve ser feito o rollback.

podemos definir as características da transação e definir quais têm mais prioridade entre e sobre os outros.
Permite assim controlar o acesso simultâneo por vários utilizadores

Se for feito rollback as operações desfeitas são apenas as que se encontram dentro da transação

Características das Transações
- Isolamento -> com vários clientes a operar sobre os dados devemos garantir a exclusão mútua entre operações.

- ver mais características

Controle de Concorrência

Escalonamento → temos de evitar ~~esse~~ inconsistência de dados (↳ Leituras perdidas)

Leituras Sujas → falhas nas transações pode causar estas situações pois uma falha irá originar rollback limpando todos os dados anteriores mas se esse rollback for feito depois de outras transação terem operado sobre esse dados irá levar a inconsistência de dados.

Métodos :

- mecanismos de locking } preventivos
- " de etiquetas }
- Métodos optimistas → ~~não~~ partem do princípio que as interferências são raras

Etiquetagem → Cada tran é etiquetada, quando há conflitos é voltam para a fila de espera

Locking → bloqueia o acesso ao registo quando precisa, em situações de deadlock o sistema liberta os recursos ~~e as transa~~

Recuperação de falhas

Todos os sistemas computacionais estão sujeitos a falhas

Ex: - durante a escrita o sistema vai a baixo
- O disco estraga-se por alguma razão

~~Falta~~ a luz hospital se a base de dados vai a baixo pode morrer pessoas e trazer-nos implicações legais

Gravidade : falha no transação → menos graves
Perda da base de dados → grave

As menos graves são tratadas pelo próprio sistema

Abort em cascata → perigo de perda grave de informação

Backups → Cópias de segurança em caso de perda de informação

Desvantagem : só possuiu info até ao ponto onde foi feito o backup.

Deve-se fazer backups com regularidade mas não pesados e demorados

Podem ser feitos em horas mortas da atividade, é preferível

Transaction Logs → registo todas as operações realizadas na base de dados, este registo não deve estar no mesmo disco que a base de dados

Ex: Em caso de falha do disco podemos ir buscar o backup e de seguida poderemos fazer o rollForword até obter os dados antes da falha.

usando os before image e after image verificamos e os dois alterações feitas antes e depois

Check point → demoreom o momento até onde é feito o rollback ou rollforword

Controle de Concorrência

Escalonamento → temos de evitar ~~esse~~ inconsistência de dados (↳ Leituras perdidas)

Leituras sujas → falhas nas transações pode causar estas situações pois uma falha irá originar rollback limpando todos os dados anteriores mas se esse rollback for feito depois de outras transação terem operado sobre esse dados irá levar a inconsistência de dados.

Métodos:

- mecanismos de locking
- " de etiquetas
} preventivos

- Métodos optimistas → ~~parte~~ parte do princípio que as interferências ~~é~~ são raras

Etiquetagem → Cada trans é etiquetada, quando há conflitos é voltam para a fila de espera

Locking → bloqueia o acesso ao registo quando precisa, em situações de deadlock o sistema liberta os recursos ~~e os transa~~

Recuperação de falhas

Todos os sistemas computacionais estão sujeitos a falhas

Ex: - durante a escrita o sistema vai a baixo
- O disco estraga-se por alguma razão

~~Para~~ Num hospital se a base de dados vai a baixo podem morrer pessoas e trazer-nos implicações legais

Gravidade: falha no transação → erros graves
Perda da base de dados → grave

As erros graves ~~é~~ tratados pelo próprio sistema

Abort em cascata → perigo de perda grave de informação

Backups → Cópias de segurança em caso de perda de informação

Desvantagem: só possuí-mos inf. até ao ponto onde foi feito o backup.

Deve-se fazer backups com regularidade mas ~~é~~ pesados e demorados

Podem ser feitos em horas mortas da atividade, é preferível

Transaction Logs → regista todas as operações realizadas na base de dados, este registo ~~é~~ deve estar no mesmo disco que a base de dados

Ex: Em caso de falha do disco podemos ir buscar o backup e de seguida podemos fazer o rollForword até obter os dados antes da falha.

usando os before image e after image verificamos as alterações feitas antes e depois

Check point → demoram o momento até onde é feito o rollback ou rollforword

Save ~~point~~ & policies

Segurança

Tal como num sistema operativo no sql também vários objetos com permissões diferentes

No sql há autenticação, podemos criar contas no sql ~~e~~ no windows, só para logins no ~~com~~ sql server

Os login podem estar associados a regras, um administrador ~~po~~ tem permissões adicionais, ao contrário do user normal

~~Esses~~ Esses utilizadores têm acesso a base de dados distintas

No managment podemos observar ~~vários~~ vários logins e regras dos servidores, estas regras definem quais previlegios para tipo de utilizador tem. Um utilizador pode pertencer a vários grupos.

Escolhendo uma ~~bad~~ base de dados também temos os users e as regras

Podemos criar Login mas deveremos usar um sp- que serve especificamente para isso

Cada base de dados tem informação e também tem metadados ~~q~~ relacionados com ela.

Temos uma grande quantidade views na master, ~~e~~ e em qualquer base dados.

Podemos consultar informação como Tabelas criadas, foreign Keys, indices, sp's, views, etc.



Na tabela Local-de-Estágio inserir ~~uma~~ coluna nome ✓

Listagem de estágios feitos, com dados de ~~aluno~~ aluno, ~~especialidade~~ especialidade do estágio

Tabela Aluno-Realizou-estágio

Realização estágio

Criar view

Na tabela estágio alt. coluna especialidade ✓
~~q~~ contendo ~~está~~ na tabela Realização-estágio ✓

~~Nome~~ ~~Ano~~
Local ✓
Localização ✓
Especialidade ✓

Professor
Ano
Especialidade
Telefone
Sal

Data Início ✓
Data fim ✓
Contacto Prof ✓

Nome ✓
Apelido ✓ e-mail ✓
Telefone ✓
Ano ✓
Curso ✓
Data conclusão ✓

Criar trigger para o insert de cp escolhas
verifica se só tem um tuplo

criar pass de user hasho no lado da aplicação

Através das views podemos ver colunas, filtro-las por tabelas, etc

Para uma aplicação que faça acesso a base dados há um user que terá certas permissões

É mais fácil criar perfis com permissões pre-definidas e de seguida adicionar o utilizador a certos perfis. role

Temos de ter cuidado pois podemos ter permissões para executar um sp mas dentro desse sp há operação que n tenho permissões para executar

SQL Injection

Ñ podemos fazer a manipulação de código direto do lado da aplicação misturado com objetos do código do outro lado, torna a base de dados vulnerável a ataques

Ver possíveis ataques nos slides

É possível evitar algumas ataque retirando permissões de acesso mas acabaria por tirar também funcionalidades, para isso devemos usar outras técnicas.

Desconfiar sempre nos dados inseridos pelo utilizador → limpeza de caracteres verificando caracteres perigosos

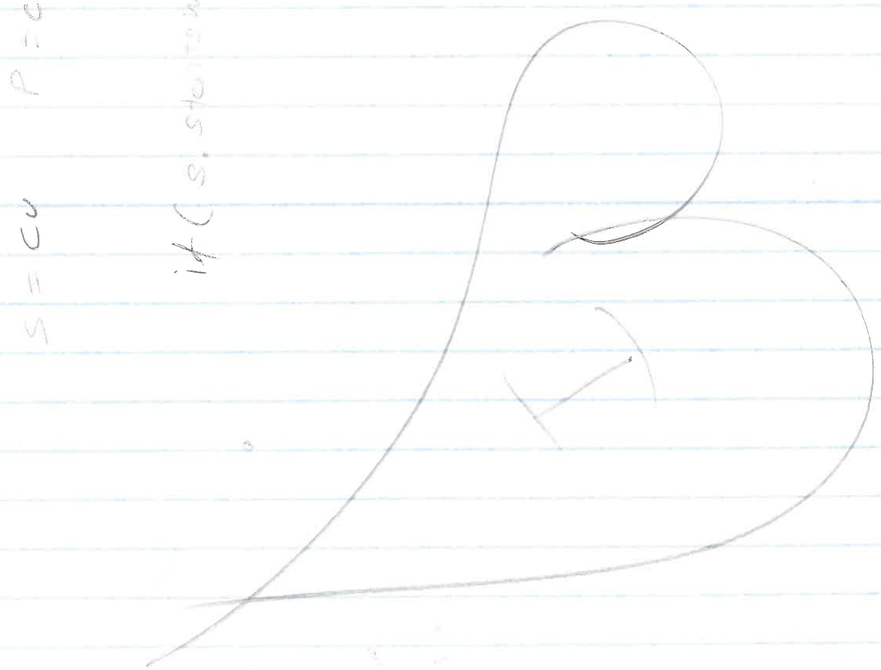
Nunca utilizar SQL Dinâmico

Criar trigger para [illegible]

Aviso tool! → vem estágios feitos atrasado e DDL anotação

Criar Trigger verifica se Aluno é professor, orientad.

Criar trigger para verifica se [illegible]



Criar Trigger na tabela ~~estágio~~ Aluno-Pedido, ~~apenas de~~ só
deixar criar depois do estágio terminar →

Criar SP-Auditoria aluno ✓

No SQL Server podemos associar os objetos a 5 vistas todos,
é mais eficiente associar a um role

Também possível criar sp's com permissões apenas
para determinados users ↳ UDF e Triggers

CREATE PROC ...

WITH EXECUTE AS SELF
↳ permissões iguais a quem criou
CALLER
↳ permissões de quem chama
CLOSE → utilizador

Isto é útil pois e.g. ..., pode ter instruções que quem
chamou o sp não tem e assim é interrompido → perigoso

Também é possível cifrar atributos

Análise de base de dados

Critérios Informais - clareza dos atributos
- verificar se existe ~~dúvida~~ uma
semântica entre os atributos e o
relação

Objetivo → redução de nulls
↳ para reais termos tuplos que já se aplicam
à relação, deve-se criar uma outra tabela

Dependências formais

$Y \rightarrow Y$

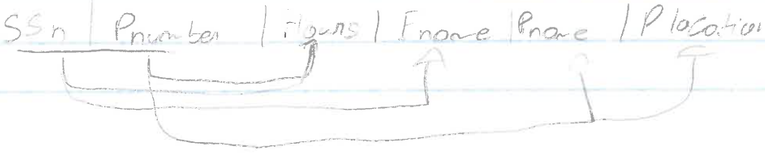
↳ x define unicamente Y

Dependência parcial → atributo depende parcialmente
da chave
" Total → atributo que depende totalmente
da chave da relação
" Transitivo → um atributo que não chave
depende de outro e que não é chave e vice-versa

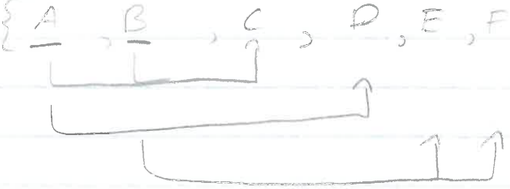
Normalização → RD Redundância

1FN → Atomicidade dos atributos, não há obtenção
Non nested Relations
Relações aninhadas

2FN → Eliminar dependências parciais



$R = \{ \underline{A}, \underline{B}, C, D, E, F \}$



$R_1 = \{ \underline{A}, C \}$

$R_2 = \{ \underline{B}, E, F \}$

$R_3 = \{ \underline{A}, \underline{B}, C \}$

3 FN → Eliminar as Dependências transitivas

$R = \{ A, \underline{B}, C, D, E, F, G \}$

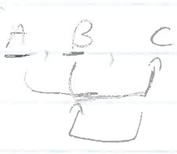


$R_1 = \{ A, \underline{B}, C, D, E \}$

$R_2 = \{ \underline{E}, F, G \}$

BC FN → Por vezes todos os atributos dependem do chave mas parte dessa chave depende de um atributos

$R = \{ \underline{A}, \underline{B}, C \}$



$R_1 = \{ \underline{B}, \underline{C} \}$ perde os DF

$R_2 = \{ \underline{A}, C \}$

Deve existir equilíbrio

B

$R = \{ A, B, C, D, F, \underline{G}, \underline{H}, \underline{I} \}$



1 FN

$R_1 = \{ \underline{A}, B, C, D, F \}$

$R_2 = \{ \underline{A}, \underline{G}, H, I \}$



2 FN

$R_1 = \{ \underline{A}, B, C, D, F \}$



$R_2 = \{ \underline{A}, \underline{G} \}$

$R_3 = \{ \underline{G}, H, I \}$

3 FN

$R_1 = \{ \underline{A}, B, F \}$

$R_2 = \{ \underline{A}, \underline{G} \}$

$R_3 = \{ \underline{G}, H, I \}$

$R_4 = \{ \underline{B}, C, D \}$

doc Alunos

Single-Level Ordered

Armazena cada valor do atributo indexado

Primário → index sobre a chave primária
clustered → " " um atributo que não é chave primária → conjunto de tuplos

Secundário → índice para uma tabela de ~~índices~~ chaves
únicos
primary index em ~~multiplos~~ níveis

Multi-level → índice para um índice de tabelas
na tentativa de reduzir a complexidade
de $log_2 b_i$ para $\approx log_{10} b_i$

Solução de índices compromisso entre perceber se
irão haver muitas pesquisas ~~com~~ num determinado atributo
e funções nesse mesmo atributo e perceber se irão haver
muitas modificações nesse mesmo atributo

Se a pesquisa é feita por um atributo que não a PK ent
então talvez seja mais eficiente criar um índice

clustered
uma tabela de PKD'S aponta para uma tabela de
de índices ou para uma tabela onde os ~~dados~~ dados
associados à opela chave se encontram

Non-clustered
A partir de dados que não as PK são contidos
tabelas de paginação com a PK
adjacente aos dados

Cada índice corresponde para um instance nesse para
listagens de alunos
~~caso~~
Criar índice para o nome do aluno ✓ ~~pesquisa~~

SQL Server apenas possui índices clustered e non-clustered

~~Que~~ podemos alterar o Fill Factor de maneira a
o que o page-split ocorra com menos frequência,
evitando assim as operações que são muito exigentes de
CPU.
Também deve ser feita a desfragmentação das tabelas de
índice regularmente, para eliminar espaços vazios que possam
surgir após operações de remoção ou page split

ALTER INDEX "nome" ON tabela REORGANIZE
Podemos também alterar o fillfactor para algo mais
adequado à situação

esta desfragmentação
tem em conta o fillfactor

Programação em SQL
Case
WHEN 1 THEN 'Mau'
WHEN 2 THEN 'Sulco'
ELSE ~~&~~ 'desconhecido'
END

visíveis nos inner level

É possível declarar tabelas temporárias #Table e tabelas temporárias globais ## são eliminadas quando o último sessão acaba

As tabelas também podem variáveis mas têm um scope mais limitado, terminam quando o batch, function, ou stored procedure chega ao fim.
Não permite restrições de FK

Cursores → operações de que são executados sequencialmente
Ferramenta que permite percorrer os rolos retornados de uma operação SELECT

Estas operações são bastante exigentes para o motor de base de dados e devem ser evitados

Sintaxe:

DECLARE C CURSOR FAST_FORWARD
FOR SELECT * FROM

~~CURSOR OPEN~~

OPEN C; → O cursor tem de ser aberto
DECLARE @x INT
WHILE @@FETCH_STATUS = 0
BEGIN
FETCH NEXT FROM C INTO @x

CLOSE C; → Tem de ser fechado
DEALLOCATE C → Espaço é libertado

@@error → O valor de saída de cada instrução é retornado nesta variável
@@rowcount → nº de linhas afetado

RAISERROR → Retorna mensagem de erro ao utilizador

RAISERROR ('ERROR MSG', Severity Code, 1, objectos operadores)

pode ser criado SP para que não seja possível alterar ou verificar o conteúdo do SP
CREATE Procedure ...
WITH ENCRYPTION

Mais valias dos SP → É a melhor maneira de que um utilizador tem para executar operações sobre a base de dados sem ter de saber a sua estrutura interna
- A sua execução é eficiente
- Restringindo o acesso do utilizador ao SP, elimina-se o risco de manipulação

UDF's podem ser usados em Cross Apply de dados, pode-se passar parâmetros ao contrário dos views.

UDF Scalar → Retornam um valor, podem ser usadas em qualquer lado aí em Check Constraints
Não são permitidos certas operações como update à base de dados
Try catch

UDF Inline Table value → retornam tabelas, extensão de um SELECT

UDF Multi-Statement Table-valued → É possível combinar o facto de ser possível de combinar código complexo com a capacidade de retornar uma tabela

Sintaxe:

```
CREATE FUNCTION dbo.ola (@price Money)
RETURNS @Table TABLE (nome VARCHAR(100), ssn INT)
AS
BEGIN
    INSERT @Table SELECT *
        FROM x

    RETURN
END
```

```
BEGIN
    INSERT @table (Code, EffectiveDate, Price)
        SELECT Code, null, avg(Price)
        FROM ...

    RETURN
END
```

```
SELECT * FROM dbo.ola 15,6
```

Os 3 tipos de UDFs podem ser definidos com Schema Binding, o que proíbe adicionar informação a uma tabela mas não pode alterar ou eliminar → vantagem sobre sp

```
CREATE FUNCTION dbo.x (...) RETURNS ...
WITH SCHEMA BINDING
```

Triggers

Semelhantes às SP mas são ativados por certos eventos e são suportados em DDL e DML

Compromisso entre integridade e alto desempenho

No DML há dois tipos de triggers

Instead of → verificamos alguns condições e depois chamamos novamente a operação, como o trigger não é recursivo vai dois ser ativado novamente

after

pode ocorrer

que todos os dados são passados pelos constraints

```
CREATE TRIGGER nome ON Tabelx AFTER/INSTEAD OF
INSERT, DELETE, UPDATE
AS
```

pode ativar ou desativar Triggers

```
ENABLE/DISABLE TRIGGER nome ON Tabelx
```

Com os Triggers temos acesso à tabela temporária inserted onde se encontram os dados que irão ser inseridos na tabela

AFTER → podemos ter vários por tabela

INSTEAD OF → Apenas 1 por tabela

Com TRIGGERS temos acesso a dois ~~testes~~ tabelas lógicas Deleted e INSERTED

INSERT ——> inserted
DELETE -> DELETED
UPDATE -> DELETED e INSERTED

Tem scope muito limitado

Nos triggers ainda temos acesso a duas funções que nos permitem saber quais as colunas afetadas
IF UPDATE (Column)

Columns_updated -> retorna bitmap

Limitações -> Não são permitidos
CREATE, ALTER, DROP,
etc

Transação -> conjunto de operações de escrita e leitura dos dados da base de dados

Por defeito todos os comandos Sql são transacionais

Propriedades de uma transação

Atomicidade -> Ou a operação é completa ou então não é feita nada

Integridade -> Após todas as operações os estados de integridade têm que se manter.

Isolamento -> Cada transação deve ser executado de forma única e o seu acesso

o recursos deve ser feito em exclusão mútua.

Persistência: Os efeitos de uma transação são visíveis pelas as outras transações, falhas anteriores não devem afetar operações futuras

Isolamento -> Recurso aos técnicas de scheduling que definem a ordem pela qual são feitas as operações

Escalonamento Serializado - uma transação é executada de cada vez, pouco eficiente

Concorrente Serializado - As transações são executadas em simultâneo desde que não existe interferência na execução de ambos. Ex: Enquanto operações via a fazer operações de I/O lentas, outra pode executar as operações mais eficiente

O escalonamento concorrente pode levar a vários problemas de como é caso das leituras sujas, ou atualizações perdidas, isto deve-se ao facto de operações distintas fazerem uso do mesmo recurso.

Atualização perdida -> uma transação escreve informação que já tinha sido lida e escrita por outra

leitura suja -> uma transação lê um valor, que, devido a alguma situação já foi alterado por uma operação de rollback, tendo o seu valor e consequentemente a uma inconsistência de dados por parte da transição transação que o leu antes do rollback

Métodos de controlo de concorrência

Mecanismos de Etiquetagem - Quando a transação inicia é-lhe atribuída uma etiqueta sequencial, quando acede a um elemento modificado, se outra tenta aceder e a ne é superior ao seu esse acesso retira-se outro atrasa

Mecanismos de Locking - uma transação só acede a um elemento se esse elemento estiver desbloqueado, quando a tem na sua posse esse elemento é bloqueado, o para além de locks de acesso também há locks de leitura e escrita. → obriga a implementação de políticas de prevenção de deadlock

Ainda há técnicas optimistas que consideram que existem poucas intersecções entre as transações

Recuperação de falhas

Todos os sistemas estão sujeitos a falhas, e o SGBD não é exceção e por isso deve estar preparado

Gravidade

- ~~0~~ - graves → falhas nas transações → Recorrem de Escalonamento
- t graves → Perda parcial ou total de dados → Backups

Transaction log

Com as técnicas de recuperação por escalonamento

Temos recuperáveis, e não recuperáveis

Com aborts em cascata e ~~Com~~ Sem aborts em cascata

Escritos e não escritos

Recuperável e não recuperável → um escalonamento é recuperável se nenhuma Trans. for concluída até que todos os outros que operaram nos seus também tenham sido concluídos

Recuperáveis

Com e sem aborts em cascata → aborts em cascata devem ser evitados, ~~pois~~ tornam o processo de recuperação pouco eficiente

Para evitar é uma transação só pode ler um objeto, ~~mesmo~~ depois de que esteja a ser usado por mais, se estiver só lê pelo o seu execução

Recuperável, sem aborts em cascata, escrito-se um certo transação só escreve num elemento após outra transação que esteja a usar esse mesmo elemento só feito ~~todas~~ tenham concluído as sua execução.

Backups

Cópias de segurança efetuadas com regularidade, só pode recuperar até determinado ponto.

Como as operações de backup são exigentes para o sistema, apenas devem ser feitas em momentos em que o SGBD esteja a ser pouco usado.

Transaction logs → Registam todas as operações feitas na BD, incluindo commits.

Por questões de eficiência de I/O os dados são escritos em buffers e só depois em disco

Transaction logs separa-se pelo disco e pela memória, os registos das operações têm de ser escritos a disco antes do registo do commit

Também guardam uma imagem dos dados antes e depois das alterações

Em caso de falha os backup e os transaction logs. Em caso de falha faz-se o roll forward recuperando a base dados com o backup e de seguida → usando os transaction logs com as after e before images

Em caso de falhas nas transações basta usar a before-image capturada antes da transação

Em caso de falha do sistema operativo ou do SGBD, também se faz o rollback até ao último momento em que a base dados deixou de ser integra

↓

Até que ponto?

Um simples rollback não deve obrigar a base de Dados a fazer rollback até ao ponto de backup, pode estar afastado

Existem mecanismos de savepoint que registam até que ponto se deve fazer o rollback

Consequências dos mecanismos de recuperação de falhas

- Necessidade de mais espaço no disco
- Mais leituras e escritas
- Mais uso de CPU

Aspetos de segurança

Para um login ter acesso a uma bd tem de estar associado a um userID, pode um login pode estar associado a muitos userID